

Cuprins

1	Introducere	1
1.1	Contextul actual și motivația alegerii temei	1
1.2	Repere comparative și delimitarea problemei	2
1.3	Prezentarea generală a aplicației WhereWeFishin	2
1.4	Obiectivele lucrării și contribuțiile originale	3
1.5	Structura lucrării	4
2	Fundamente teoretice și tehnologiile utilizate	5
2.1	Arhitectura generală a sistemului	5
2.2	Modelul de date și entitățile de domeniu	6
2.3	Harta interactivă și componenta geospațială	6
2.4	Rezervări și sesiuni de pescuit	7
2.5	Autentificare, autorizare și securitate	8
2.6	Analiza imaginilor și videoclipurilor cu inteligență artificială	9
2.7	Servicii auxiliare și integrarea cu sisteme externe	9
2.8	Sinteza capitolului	10
3	Analiza cerințelor și proiectarea sistemului	11
3.1	Actorii sistemului și obiectivele operaționale	11
3.2	Cerințe funcționale	12
3.3	Cerințe non-funcționale și reguli de business	13
3.4	Proiectarea de ansamblu a sistemului	13
3.5	Modelul de domeniu și relațiile dintre entități	14
3.6	Fluxuri esențiale și diagrame	15
3.7	Sinteza capitolului	16
4	Implementare și contribuții personale	17
4.1	Arhitectura implementată a soluției	17
4.2	Implementarea backend-ului	18
4.2.1	Autentificare, autorizare și controlul accesului	18
4.2.2	Locuri de pescuit, pontoane și organizarea resurselor	18
4.2.3	Workflow-ul de aplicare pentru manager	19
4.2.4	Rezervări, sesiuni de pescuit și integrarea plății	19
4.2.5	Angajați, recenzii și servicii auxiliare	20
4.3	Implementarea frontend-ului	20
4.3.1	Structura aplicației SPA și controlul accesului în interfață	20
4.3.2	Harta interactivă și explorarea locurilor de pescuit	21
4.3.3	Coșul, calendarul și integrarea Stripe în interfață	21

4.3.4	Wizard-ul de aplicare și dashboard-ul operational	22
4.3.5	Verificarea prin cod QR și funcțiile administrative	22
4.4	Implementarea sistemului de analiză media	22
4.4.1	Integrarea dintre backend și serviciul Python	24
4.4.2	Pipeline-ul video: detecție, tracking și post-procesare	24
4.4.3	Afișarea rezultatelor în frontend și tratarea stării asincrone	24
4.5	Infrastructura de deployment	25
4.5.1	Containerizarea cu Docker și build-uri multi-etapă	25
4.5.2	Rate limiting și securitatea pipeline-ului HTTP	26
4.5.3	Health check-uri și reziliența serviciilor	28
4.5.4	Expunerea publică prin Cloudflare Tunnel	28
4.6	Contribuțiile principale în perspectivă de ansamblu	29
4.7	Sinteza capitolului	29
5	Modelul de inteligență artificială și setul de date	31
5.1	Contextul și motivația unui set de date dedicat	31
5.2	Colectarea și structura setului de date	31
5.3	Anotarea și preprocesarea imaginilor	32
5.4	Antrenarea modelului	32
5.5	Evaluarea performanței modelului	33
5.6	Integrarea modelului antrenat în producție	34
5.7	Limitări și observații ale modelului	35
5.8	Sinteza capitolului	35
6	Interfața utilizatorului și experiența de utilizare	36
6.1	Principii de organizare a interfeței	36
6.2	Experiența utilizatorului obișnuit	37
6.3	Interfața managerului și dashboard-ul operațional	37
6.4	Experiența angajatului și verificarea prin cod QR	38
6.5	Panoul administrativ	38
6.6	Gestionarea autentificării și comunicarea cu API-ul	39
6.7	Adaptabilitate și considerații de utilizare mobilă	39
6.8	Sinteza capitolului	40
7	Testare și evaluare	41
7.1	Strategia de testare și infrastructura utilizată	41
7.2	Rezultatele rulării suitei de teste	42
7.3	Testarea unitară și validarea datelor	43
7.4	Testarea controllerelor și a comportamentului API	43
7.5	Testarea de integrare a fluxurilor critice	43
7.6	Validarea operațională a comportamentului sistemului	44
7.7	Acoperire actuală, limitări și direcții de extindere	44
7.8	Sinteza capitolului	44
8	Securitate și infrastructură de deployment	46
8.1	Arhitectura de securitate în straturi	46
8.2	Autentificarea prin JSON Web Tokens	46
8.3	Autorizare pe roluri și proprietate	47
8.4	Protecția fluxurilor sensibile	48

8.5	Izolarea componentelor și securitatea comunicației interne	49
8.6	Containerizarea aplicației cu Docker	49
8.7	Gestionarea configurației pe medii de execuție	51
8.8	Sinteza capitolului	51
9	Concluzii și perspective de dezvoltare	52
9.1	Concluzii generale	52
9.2	Contribuțiile principale ale lucrării	53
9.3	Limitele actuale ale soluției	53
9.4	Perspective de dezvoltare	54
9.5	Încheiere	54

Lista figurilor și diagramelor

1.1	Cele patru roluri ale utilizatorilor și principalele funcții disponibile	3
2.1	Principalele entități și relații din modelul de date	7
3.1	Diagrama de cazuri de utilizare (simplificată)	12
3.2	Arhitectura de ansamblu a aplicației WhereWeFishin	14
3.3	Diagrama claselor principale cu atributele entităților de domeniu	15
3.4	Fluxul principal al unei rezervări și al validării sesiunii de pescuit	15
3.5	Fluxul de analiză media: de la upload până la afișarea rezultatelor . . .	15
4.1	Fluxul de autentificare bazat pe tokenuri JWT	18
4.2	Serviciile Docker și dependențele dintre ele	25
4.3	Distribuția contribuțiilor personale pe zone funcționale ale platformei . .	29
5.1	Relația dintre metricile principale de evaluare a modelului	33
5.2	Precizie, Recall și mAP@0.5 per clasă pe setul de test	34
7.1	Distribuția estimată a testelor pe categorii în proiectul backend	42
8.1	Arhitectura containerizată a aplicației WhereWeFishin	50

Capitolul 1

Introducere

1.1 Contextul actual și motivația alegerii temei

Pescuitul a fost mereu mai mult decât o pasiune pentru mine, e o parte din cine sunt. Totul a început de la tata. El m-a luat cu el de când eram doar un copil și tot el a avut răbdarea să mă învețe totul. Îmi amintesc și acum diminețile acelea în care mă trezeam cu noaptea-n cap, plin de entuziasm așteptând să ajung la baltă. Nu înțelegeam pe atunci de ce mă atrage atât de mult, știam doar că acolo mă simțeam bine.

Odată cu anii studenției, am început să merg singur. Am vrut să descopăr locuri noi, să evadesc din agitația orelor și a orașului. Dar, pe măsură ce încercam să fac asta, m-am lovit de o grămadă de detalii obositoare care stricau toată magia, telefoane date în stânga și-n dreapta, zeci de întrebări ca să găsesc măcar un ponton liber, rezervări făcute pe genunchi sau pur și simplu drumuri făcute pentru nimic.

La un moment dat m-am întrebat: oare de ce un lucru atât de simplu trebuie să fie atât de complicat și stresant de rezervat?

De la această întrebare a pornit ideea lucrării de față. Digitalizarea a schimbat mult felul în care oamenii interacționează cu serviciile din jur. Rezervările de hotel, comenzile de mâncare sau cumpărăturile online se fac acum în câteva minute, de pe telefon. Chiar și activitățile recreative au început să urmeze același drum. Pescuitul recreativ este unul dintre puținele domenii în care procesele rămân fragmentate și, în mare parte, neautomatizate.

Un pescar care vrea să găsească un loc nou are câteva variante: caută pe grupuri de Facebook, sună direct la locație sau merge fizic să se intereseze. Dacă vrea să rezerve un ponton anume, de cele mai multe ori o face tot prin telefon sau mesaj. Administratorul locației ține evidența rezervărilor cu bonuri, chitanțe și notițe scrise de mână. Erorile apar frecvent, iar comunicarea este greoaie.

Această lucrare pornește de la nevoia de a simplifica și integra aceste procese. O aplicație dedicată pescuitului recreativ nu mai trebuie să fie un simplu site cu locații pe hartă. Poate fi o platformă completă: utilizatorul găsește un loc de pescuit, rezervă un ponton, plătește online și primește o confirmare. Iar administratorul locației are un panou de control prin care vede rezervările, gestionează angajații și monitorizează activitatea.

WhereWeFishin a fost gândit exact din această idee: o platformă care reunește descoperirea locurilor de pescuit, rezervarea, administrarea operațională și analiza automată a fișierelor media, totul într-un singur sistem.

Proiectul se înscrie într-un trend mai larg, în care platformele digitale nu mai servesc

doar informarea sau comerțul electronic, ci și coordonarea unor activități recreative cu logistică proprie: rezervarea terenurilor de sport, planificarea excursiilor sau administrarea spațiilor de camping. Pescuitul recreativ are particularități specifice: locuri cu acces limitat, pontoane cu disponibilitate variabilă, sezonabilitate pronunțată, care necesită un sistem adaptat contextului, nu un instrument generic de rezervări.

O platformă de acest tip este utilă nu doar pescarilor, ci și administratorilor locațiilor: vizibilitate online mai mare, mai puțin timp pierdut cu confirmări telefonice, o evidență clară a rezervărilor și posibilitatea de a delega operațiile de verificare angajaților. Digitalizarea procesului aduce valoare reală ambelor părți implicate.

1.2 Repere comparative și delimitarea problemei

Există deja câteva aplicații care se ocupă parțial de activitățile legate de pescuitul recreativ. Fishbrain, FishAngler și ANGLR sunt printre cele mai cunoscute și pun accentul pe jurnalizarea capturilor, pe comunitate și pe identificarea locurilor de pescuit din experiența altor pescari [3–5]. Sunt utile, dar se concentrează aproape exclusiv pe experiența individuală a pescarului.

Fishbrain, de exemplu, pune accent pe rețeaua socială: localizarea capturilor pe hartă, comentarii ale altor pescari, predicții meteorologice și distribuția speciilor în funcție de zonă. FishAngler adaugă posibilitatea de a documenta sesiunile cu detalii despre condițiile de pescuit și de a crea trasee personalizate. ANGLR se concentrează pe jurnalizarea automatizată prin GPS, înregistrând traseele și capturile fără intervenție manuală. Toate trei sunt produse valoroase pentru o anumită categorie de utilizatori, dar niciunul nu adresează în mod direct administrarea unei locații de pescuit sau rezervarea unui ponton specific.

Ce lipsește din aproape toate aceste aplicații este latura operațională: rezervarea controlată a unui ponton, validarea prezenței prin mijloace digitale, separarea clară a responsabilităților pe roluri (utilizator, angajat, manager, administrator) și administrarea propriu-zisă a unei locații de pescuit. Acestea sunt exact procesele pe care le abordează WhereWeFishin.

Un al doilea element de diferențiere este integrarea analizei automate a fișierelor media. Utilizatorul poate încărca o fotografie sau un videoclip cu o captură, iar aplicația folosește modele de inteligență artificială pentru a identifica specia de pește. Puține soluții combină, într-un flux coerent, o platformă web cu procesare automată a imaginilor și videoclipurilor.

Concluzia analizei comparative este că WhereWeFishin nu vrea să fie o alternativă la aplicațiile sociale existente, ci acoperă un gol funcțional real: o platformă care deservește atât pescarul obișnuit, cât și administratorul locației, cu instrumente potrivite pentru fiecare.

1.3 Prezentarea generală a aplicației WhereWeFishin

WhereWeFishin este o aplicație web construită ca o platformă integrată pentru descoperirea și administrarea experiențelor de pescuit. Utilizatorul poate găsi locuri de pescuit pe o hartă interactivă, poate rezerva un ponton, poate vedea istoricul activității sale și poate încărca imagini sau videoclipuri pentru analiză automată.

Aplicația este structurată pe patru componente principale. Interfața cu utilizatorul este o aplicație web de tip SPA construită cu Angular [6], cu o hartă interactivă bazată pe Leaflet [12]. Backend-ul este un API HTTP realizat cu ASP.NET Core [7], care gestionează logica de business, autentificarea și comunicarea cu serviciile externe. Datele sunt stocate într-o bază de date relațională accesată prin Entity Framework Core [9, 15]. Analiza imaginilor și videoclipurilor este realizată de un microserviciu separat, scris în Python cu Flask [11].

Aplicația susține patru roluri de utilizator, fiecare cu un set clar de funcții. Figura 1.1 ilustrează această structură.

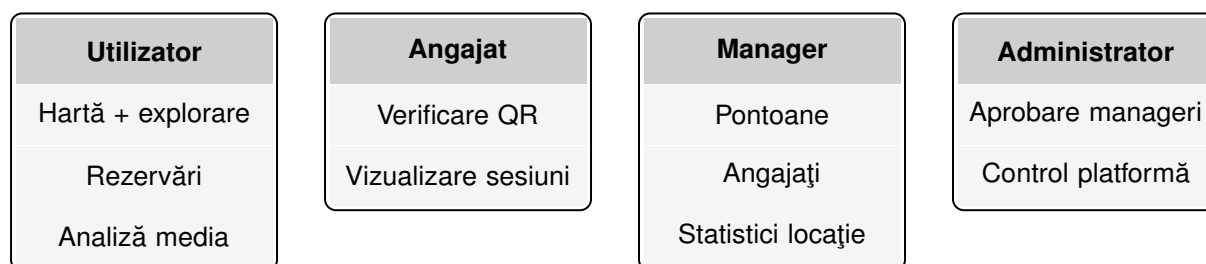


Figura 1.1: Cele patru roluri ale utilizatorilor și principalele funcții disponibile

Un flux central al aplicației este rezervarea. Utilizatorul selectează o locație, alege un ponton și un interval orar, iar sistemul calculează costul și inițiază plata prin Stripe [16]. La finalul procesului, sesiunea de pescuit poate fi validată la fața locului printr-un cod QR și un token unic.

Componenta de analiză media permite încărcarea de imagini și videoclipuri, care sunt trimise automat către microserviciul Python. Acesta folosește modele YOLO pentru detecția peștilor în imagini și ByteTrack pentru urmărirea lor în secvențe video [17, 23, 26]. Rezultatele sunt stocate și afișate utilizatorului în interfață.

Harta interactivă nu este doar un element decorativ. Ea funcționează ca punct central de navigare, prin care utilizatorul descoperă locațiile, iar managerul delimitează vizual zonele rezervabile.

Platforma este accesibilă public la adresa <https://wherewefishin.uk>. Întrucât infrastructura este găzduită pe un server personal, disponibilitatea aplicației poate varia: este posibil ca la momentul accesării serverul să fie oprit.

1.4 Obiectivele lucrării și contribuțiile originale

Obiectivul principal al lucrării este proiectarea și implementarea unei platforme care să digitalizeze procesele asociate pescuitului recreativ, de la găsirea locației și rezervare până la administrare și analiză media.

Obiectivele specifice sunt:

1. definirea unei arhitecturi software clare, cu separarea responsabilităților între frontend, backend, persistență și microserviciul de analiză media;
2. modelarea unor fluxuri reale de utilizare: rezervare, validare, administrare și gestionare pe roluri;
3. integrarea unui modul de analiză media bazat pe inteligență artificială într-un sistem orientat spre utilizare curentă;

4. construirea unei soluții extensibile, care să poată fi îmbunătățită ulterior fără rescrierea arhitecturii.

Contribuțiile originale ale acestei lucrări sunt:

1. integrarea într-o singură platformă a funcțiilor de descoperire, rezervare, administrare și analiză media;
2. modelul de roluri cu patru niveluri, fiecare cu permisiuni bine definite;
3. harta interactivă ca element central al navigării, inclusiv pentru delimitarea vizuală a pontoanelor;
4. mecanismul de verificare prin cod QR și token, pentru confirmarea prezenței la locație;
5. microserviciul Python cu YOLO și ByteTrack, integrat controlat în fluxul aplicației;
6. arhitectura modulară, potrivită pentru întreținere, testare și extindere ulterioară.

Valoarea acestor contribuții nu stă în fiecare element separat, ci în combinarea lor. O hartă interactivă sau un sistem de rezervări pot fi găsite și în alte aplicații. Ceea ce diferențiază WhereWeFishin este integrarea lor coerentă cu administrarea operațională și analiza AI, într-un singur sistem orientat spre un domeniu concret.

1.5 Structura lucrării

Lucrarea este organizată în opt capitole. Primul (cel de față) prezintă contextul, motivația și obiectivele proiectului.

Capitolul al doilea prezintă fundamentele teoretice și tehnologiile utilizate: arhitectura sistemului, modelul de date, componenta geospațială, fluxul de rezervare, autentificarea și analiza media bazată pe inteligență artificială.

Capitolul al treilea detaliază analiza cerințelor și proiectarea sistemului: rolurile utilizatorilor, cerințele funcționale și non-funcționale, modelul de domeniu și principalele decizii arhitecturale.

Capitolul al patrulea prezintă implementarea propriu-zisă, cu accent pe modulele backend, interfața Angular, serviciul de analiză media și contribuțiile personale din fiecare zonă.

Capitolul al cincilea descrie construirea componentei de inteligență artificială: colectarea și adnotarea setului de date propriu, antrenarea modelului YOLOv8, evaluarea performanței și integrarea modelului în serviciul Python de producție.

Capitolul al șaselea prezintă interfața utilizatorului și experiența de utilizare: organizarea interfeței pe roluri, fluxurile principale pentru fiecare tip de utilizator și considerațiile de utilizare mobilă.

Capitolul al șaptelea discută testarea și evaluarea aplicației: infrastructura de teste, rezultatele obținute și limitele actuale.

Capitolul al optulea tratează securitatea și infrastructura de deployment: autentificarea JWT, autorizarea pe roluri, protecția fluxurilor sensibile, containerizarea cu Docker și gestionarea configurației pe medii diferite.

Capitolul al nouălea sintetizează concluziile proiectului și prezintă direcțiile de extindere a platformei.

Capitolul 2

Fundamente teoretice și tehnologiile utilizate

Acest capitol prezintă baza teoretică și tehnologică pe care se sprijină aplicația WhereWeFishin. Dacă în capitolul introductiv am descris motivația și obiectivele proiectului, acum mă concentrez pe conceptele care au ghidat proiectarea și implementarea efectivă a sistemului. Sunt analizate arhitectura generală, modelul de date, componenta geospațială, fluxul de rezervare, autentificarea și componenta de analiză media bazată pe inteligență artificială.

Scopul nu este reproducerea detaliilor de cod, ci clarificarea principiilor pe care funcționează sistemul și justificarea alegerilor tehnologice. Această abordare urmează ideea clasică din ingineria software că structura unui sistem trebuie dedusă din cerințe și din separarea clară a responsabilităților [22, 25].

2.1 Arhitectura generală a sistemului

O aplicație care deservește simultan utilizatori finali, angajați și administratori are nevoie de o arhitectură bine structurată. Trebuie să gestioneze interacțiuni de interfață, validări de business, persistența datelor, comunicarea cu servicii externe și, în cazul de față, procesarea unor fișiere media. Din acest motiv, WhereWeFishin este construită ca un sistem cu mai multe componente distincte, dar integrate coerent.

Interfața cu utilizatorul este o aplicație web de tip SPA dezvoltată cu Angular [6]. Această alegere este potrivită pentru o aplicație cu navigare fluidă, componente reutilizabile, formulare complexe și actualizare dinamică a conținutului. WhereWeFishin are fluxuri variate (autentificare, hartă interactivă, rezervare, administrare, upload de fișiere), iar o interfață bazată pe componente permite separarea clară a responsabilităților.

Backend-ul este un API HTTP realizat cu ASP.NET Core [7]. El centralizează logica de business, aplică regulile de validare, controlează accesul la resurse și expune servicii pentru interfața web. ASP.NET Core oferă suport solid pentru autentificare, autorizare, middleware și interoperabilitate cu servicii externe. Stilul arhitectural adoptat este REST, larg utilizat în aplicațiile web moderne [19].

Persistența datelor se face printr-o bază de date relațională, accesată prin Entity Framework Core [9, 15]. Aceste instrumente mapează entitățile domeniului la tabele și permit interogări clare și stabile. Alegerea unui model relațional este justificată de numărul mare de relații dintre date: utilizatori, locuri de pescuit, pontoane, sesiuni de

pescuit, recenzii și rezultate ale analizelor media.

Un element distinctiv al arhitecturii este microserviciul separat pentru analiza imaginilor și videoclipurilor, implementat în Python cu Flask [11]. Separarea sa de API-ul principal are o justificare practică: procesarea media implică biblioteci specializate, cerințe diferite de execuție și timpi mai lungi de răspuns. Prin izolarea acestei responsabilități, se reduce riscul ca o operație costisitoare să afecteze stabilitatea backend-ului principal.

Modul în care componentele comunică este simplu: utilizatorul interacționează cu interfața Angular, aceasta trimite cereri către API-ul .NET, iar API-ul aplică regulile de business și salvează datele în baza de date. Când un fișier media trebuie analizat, backend-ul îl trimite microserviciului Python, primește rezultatele și le stochează.

2.2 Modelul de date și entitățile de domeniu

Un sistem informatic orientat spre un domeniu concret trebuie să pornească de la o modelare corectă a obiectelor care definesc acea activitate. În cazul unei platforme pentru pescuit recreativ, modelul de date nu descrie doar utilizatori și informații generale, ci trebuie să reflecte realități operaționale: locuri de pescuit cu coordonate geografice, zone rezervabile, sesiuni planificate și fluxuri de validare.

Entitatea centrală este **locul de pescuit**. El conține informații descriptive (denumire, descriere, imagine), coordonate geografice, tarif orar și legături cu utilizatorul care l-a creat sau cu managerul care îl administrează. În jurul lui gravitează celelalte entități importante: pontoanele, recenzii, sesiunile de pescuit, angajații și repopulările cu pești.

Pontorul este o subzonă rezervabilă din interiorul unei locații. Are o dimensiune spațială precisă, descrisă prin coordonate geografice, ceea ce permite reprezentarea lui pe hartă și rezervarea explicită. Această alegere simplifică administrarea spațiului și oferă utilizatorului o imagine mai clară a ceea ce rezervă.

Sesiunea de pescuit este entitatea care materializează o rezervare. Ea leagă utilizatorul de o locație, de un ponton (dacă este cazul), de un interval temporal, de un cost și de o stare operațională. Termenul de „sesiune de pescuit” este mai precis decât „rezervare” pentru descrierea obiectului intern, deoarece include toată informația necesară gestionării operaționale a procesului.

Utilizatorul este prezent în aproape toate fluxurile platformei: inițiază rezervări, lasă recenzii, poate deveni manager. **Recenziile** oferă feedback despre locații. **Repopulările cu pești** descriu acțiuni administrative specifice domeniului. **Analizele media** stochează rezultatele generate de microserviciul Python.

Figura 2.1 prezintă principalele entități și relațiile dintre ele.

Privit în ansamblu, modelul de date răspunde nevoii de coerență între lumea reală și reprezentarea sa informatică. Utilizarea unei baze de date relaționale și a unui ORM este potrivită deoarece permite exprimarea clară a relațiilor și reduce riscul inconsistențelor [24].

2.3 Harta interactivă și componenta geospațială

În domeniul pescuitului recreativ, poziționarea geografică nu este un detaliu secundar. Utilizatorul nu caută doar un nume de locație, ci vrea să vadă unde se află aceasta, ce locuri există în apropiere și cum este organizat spațiul.

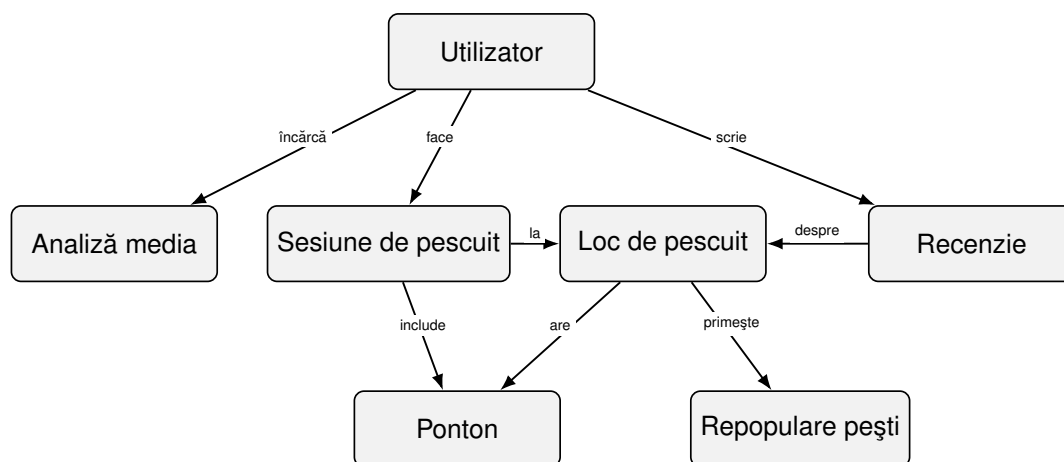


Figura 2.1: Principalele entități și relații din modelul de date

Aplicația folosește biblioteca Leaflet pentru harta interactivă [12]. Aceasta este ușoară, bine documentată, compatibilă cu dispozitivele mobile și suportă marcarea locațiilor, desenarea zonelor, ferestre informative și actualizare dinamică a conținutului. Locațiile sunt descrise prin coordonate de latitudine și longitudine, suficiente pentru afișarea pe hartă și pentru operații de tip centrare sau zoom.

Harta funcționează în două scenarii principale. Primul este explorarea: utilizatorul caută un loc de pescuit, vede pozițiile disponibile pe hartă și decide ce vrea să rezerve. Al doilea este administrarea: managerul delimitează sau actualizează vizual zona unui ponton, controlând cum apare aceasta în interfața publică.

Un aspect important este că pontonul nu este tratat doar ca o etichetă, ci ca o zonă cu poziționare geografică explicită. Utilizatorul poate înțelege mai bine ce rezervă, iar managerul poate controla mai precis împărțirea spațiului. Într-un domeniu în care proximitatea față de apă sau organizarea pe mal influențează decizia, o astfel de reprezentare este utilă.

2.4 Rezervări și sesiuni de pescuit

Fluxul de rezervare transformă informația statică despre o locație într-o interacțiune concretă. Utilizatorul solicită utilizarea unui loc sau a unui ponton pentru un interval de timp determinat. Rezultatul intern al acestei acțiuni este o sesiune de pescuit, entitatea care reține toate datele necesare gestionării operaționale.

Această distincție între „rezervare” (acțiunea utilizatorului) și „sesiune de pescuit” (obiectul intern al sistemului) este utilă atât tehnic, cât și pentru claritatea textului. Sistemul urmărește un obiect bine definit care leagă utilizatorul de locație, ponton, interval temporal, cost și stare operațională.

Pentru crearea corectă a unei sesiuni, sistemul verifică mai multe condiții: existența locației, apartenența pontonului la locația selectată și absența conflictelor temporale. Tratarea corectă a datei și orei este importantă mai ales când utilizatorii interacționează de pe dispozitive diferite.

Costul este calculat pe baza duratei și a tarifului orar al locației. Includerea lui în entitatea sesiunii permite păstrarea istoricului și analiza rezervărilor independent de valoarea curentă a tarifului.

Când plata online este activată, rezervarea se completează prin integrarea cu Stripe [16]. Aplicația nu implementează logica de plată de la zero, ci delegă această responsabilitate unui serviciu specializat, păstrând controlul asupra validărilor de business.

După confirmare, sesiunea intră într-un flux de validare la fața locului prin cod QR și token de verificare. Această soluție reduce dependența de confirmări verbale și creează un proces controlat, ușor de urmărit de angajați și manageri.

2.5 Autentificare, autorizare și securitate

Orice aplicație care gestionează conturi, date personale și rezervări trebuie să trateze securitatea ca o cerință de bază. La WhereWeFishin, această nevoie este accentuată de faptul că sistemul deservește categorii diferite de utilizatori, fiecare cu permisiuni distincte.

Autentificarea se bazează pe tokenuri JWT, standardizate prin RFC 7519 [1]. Acest mecanism este potrivit pentru un API consumat de o interfață SPA: permite transportul controlat al informațiilor despre utilizator și susține un model fără stare pe server, compatibil cu serviciile REST. Spre deosebire de sesiunile clasice stocate pe server, un token JWT conține toate informațiile necesare validării identității, semnat criptografic, astfel încât serverul nu trebuie să mențină nicio stare de sesiune.

Un token JWT este structurat din trei segmente: antet (algoritmul de semnare), payload (claims cu date despre utilizator: identificator, rol, email, emițător, audiență, dată expirare) și semnătură (HMAC-SHA256 aplicat pe primele două segmente). La fiecare cerere, backend-ul validează semnătura, verifică că emițătorul și audiența corespund configurației așteptate și că tokenul nu a expirat. Fereastra de toleranță pentru expirare este setată la zero, fără perioadă de grație.

Autentificarea singură nu este suficientă. Sistemul trebuie să decidă și ce poate face fiecare utilizator autentificat: aceasta este autorizarea. Un utilizator obișnuit poate face rezervări și consulta propriul istoric, dar nu poate administra locații ale altor persoane. Un manager operează numai asupra resurselor asociate locațiilor sale. ASP.NET Core oferă suport explicit pentru aceste mecanisme, inclusiv limitarea accesului la rute pe baza rolurilor [7].

Autorizarea funcționează pe două niveluri complementare. La nivel de endpoint, atributul `[Authorize]` cu specificarea rolului oprește orice cerere care nu provine de la un utilizator cu rolul respectiv, înainte ca logica de business să fie executată. La nivel de resursă, serviciile verifică explicit că utilizatorul autentificat este proprietarul resursei pe care încearcă să o modifice: un manager nu poate edita locația unui alt manager, chiar dacă ambii au același rol.

Securitatea nu se reduce la un singur mecanism. Ea include validarea datelor primite, protejarea fluxurilor sensibile și delimitarea clară a interacțiunilor dintre componente. Operațiile privind plățile sau accesul la rezultatele analizelor media sunt expuse doar în condiții bine definite. Un principiu aplicat consistent este că frontendului nu i se acordă încredere pentru decizii de securitate: orice validare vizibilă în interfață este replicată și în backend, independent.

2.6 Analiza imaginilor și videoclipurilor cu inteligență artificială

Una dintre componentele care diferențiază WhereWeFishin de o platformă obișnuită de rezervări este integrarea analizei media bazate pe inteligență artificială. Această zonă aparține domeniului viziunii artificiale, ramura AI care extrage automat informații din imagini și secvențe video [20]. Interesul principal este detectarea și identificarea speciilor de pești în fișierele încărcate de utilizator.

În analiza imaginilor statice, problema de bază este detecția obiectelor: localizarea peștelui în imagine și clasificarea sa. Familia de modele YOLO este larg utilizată în acest domeniu datorită vitezei și eficienței practice [17, 23]. Pentru WhereWeFishin, aceasta este alegerea potrivită deoarece funcționează bine atât pe imagini, cât și pe secvențe video.

Rezultatul unei detecții nu este o simplă decizie de tip „există” sau „nu există” pește. Modelul returnează poziția în imagine, gradul de încredere al clasificării și specia estimată. Aceste informații permit atât o reprezentare vizuală a rezultatului, cât și generarea unor statistici relevante.

Pentru videoclipuri, analiza devine mai complexă. Dacă fiecare cadru ar fi analizat independent, același pește ar putea fi contorizat de mai multe ori. De aceea, în WhereWeFishin este folosit mecanismul ByteTrack pentru urmărirea obiectelor în timp [26]. Acesta menține o asocierie coerentă între detecțiile succesive și reduce fragmentarea traseelor când același exemplar apare în mai multe cadre.

Procesarea video implică mai mulți pași: citirea cadrelor, analiza fiecărui cadru, corelarea temporală a detecțiilor și stocarea rezultatelor. Bibliotecile OpenCV și FFmpeg sunt folosite pentru prelucrarea cadrelor și recodarea fișierelor video [10, 13].

Rezultatul final poate include mai multe categorii de date: numărul total de detecții, specia dominantă, distribuția observațiilor și momentele în care au apărut obiectele detectate. Această bogăție de informație permite atât o prezentare intuitivă în interfață, cât și o interpretare mai detaliată.

Este important de menționat că analiza automată nu este infailibilă. Performanța depinde de calitatea imaginilor, iluminare, claritatea apei, unghiuri de filmare și gradul de adaptare al modelului la speciile urmărite. Rezultatele trebuie înțelese în raport cu limitele normale ale sistemelor de viziune artificială.

2.7 Servicii auxiliare și integrarea cu sisteme externe

Pe lângă componentele principale, platforma are nevoie și de servicii auxiliare care completează experiența utilizatorului și susțin operațiile curente.

Integrarea cu Stripe pentru plăți online reduce complexitatea internă și delegă logica sensibilă a tranzacțiilor unui serviciu specializat [16]. Aceasta permite și extinderea ulterioară a fluxului de plată fără modificarea arhitecturii principale.

Sistemul de notificări prin e-mail trimite confirmări după rezervare sau mesaje legate de operații importante. Notificările cresc claritatea interacțiunii și reduc dependența utilizatorului de a fi conectat permanent la aplicație pentru a afla rezultatul unei acțiuni.

Aceste servicii auxiliare au și un rol de integrare între module. O rezervare confirmată poate genera simultan o înregistrare internă, o notificare externă și o referință pentru validarea prin cod QR. O analiză media finalizată trebuie să fie accesibilă în interfață printr-un URL stabil. Aceste detalii nu schimbă modelul de domeniu, dar influențează

direct calitatea soluției finale.

2.8 Sinteza capitolului

Fundamentele prezentate în acest capitol conturează baza pe care funcționează WhereWeFishin. Arhitectura separată pe componente, modelul de date orientat spre entitățile reale ale domeniului, harta interactivă, fluxurile de rezervare și validare, mecanismele de securitate și componenta de analiză media nu sunt elemente independente, ci părți ale aceluiași sistem.

Înțelegerea acestor fundamente este importantă pentru capitolele următoare, deoarece explică de ce anumite decizii de proiectare și implementare au fost luate și cum contribuie ele la obținerea unei platforme coerente.

Capitolul 3

Analiza cerințelor și proiectarea sistemului

Acest capitol descrie cum am transformat fundamentele teoretice din capitolul anterior într-un model concret de cerințe și decizii de proiectare pentru WhereWeFishin. Scopul nu este descrierea codului, ci justificarea structurii sistemului prin raportare la fluxurile operaționale pe care acesta trebuie să le susțină. Analiza cerințelor și proiectarea sistemului sunt etape esențiale în ingineria software, deoarece definesc baza pe care se construiește implementarea [22, 25].

Capitolul este organizat în două direcții. Prima identifică rolurile sistemului și cerințele funcționale și non-funcționale. A doua descrie principalele decizii de proiectare și modelul de domeniu prin care aceste cerințe sunt transpuse în cod.

3.1 Actorii sistemului și obiectivele operaționale

Funcționarea WhereWeFishin presupune interacțiunea dintre patru categorii de utilizatori, fiecare cu responsabilități și drepturi diferite. Această diferențiere este importantă nu doar pentru securitate, ci și pentru corectitudinea modelului de domeniu. O platformă care deserveste simultan clienți finali, personal operațional și administratori nu poate folosi un model unic de interacțiune.

Cei patru actori principali sunt:

1. **utilizatorul obișnuit**, care explorează locurile de pescuit pe hartă, face rezervări, își urmărește istoricul, lasă recenzii și poate încărca fișiere media pentru analiză;
2. **angajatul**, care lucrează într-una sau mai multe locații și are acces la un set restrâns de funcții: verificarea sesiunilor și consultarea informațiilor relevante pentru locul de muncă;
3. **managerul**, care administrează un loc de pescuit, gestionează pontoanele, angajații și informațiile descriptive, și urmărește rezervările;
4. **administratorul sistemului**, care are o perspectivă globală, aprobă sau respinge cereri și supraveghează întreaga platformă.

Figura 3.1 prezintă o diagramă simplificată a cazurilor de utilizare, care evidențiază legătura dintre fiecare actor și funcțiile sale principale.

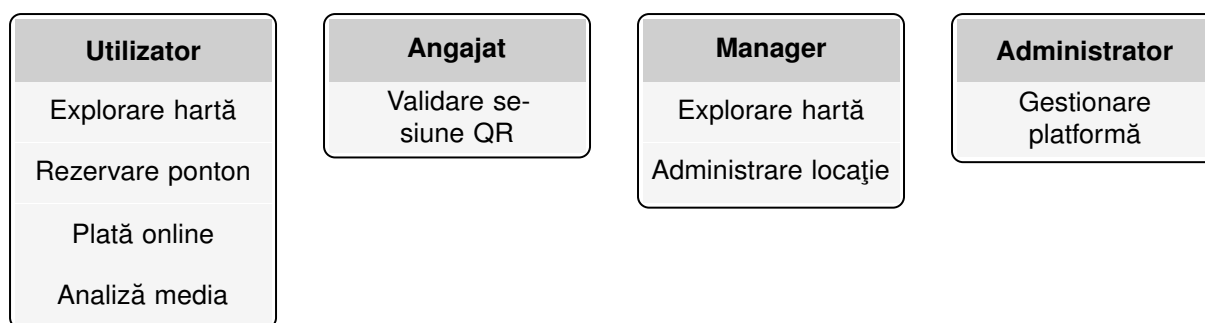


Figura 3.1: Diagrama de cazuri de utilizare (simplificată)

Această structură pe roluri generează două consecințe de proiectare. Prima este necesitatea unui mecanism de autentificare și autorizare capabil să separe clar permisiunile fiecărui actor. A doua este necesitatea modelării unor fluxuri operaționale distincte, dar interconectate: o rezervare inițiată de utilizator trebuie să poată fi verificată de angajat, iar datele administrate de manager trebuie să fie vizibile în condiții controlate pentru administrator.

3.2 Cerințe funcționale

Cerințele funcționale sunt organizate pe domenii de activitate, pentru a menține o legătură clară între nevoia reală, modelul de domeniu și componentele care trebuie să o satisfacă.

Prima categorie vizează **gestionarea identității și a accesului**. Sistemul trebuie să permită înregistrarea, autentificarea și recuperarea accesului. Platforma trebuie să distingă între roluri și să restricționeze operațiile sensibile în funcție de acestea.

A doua categorie acoperă **explorarea și administrarea locurilor de pescuit**. Utilizatorul trebuie să poată vizualiza locațiile pe hartă, să consulte detalii și să compare opțiuni. Managerul și administratorul trebuie să poată crea, actualiza și controla informațiile despre locații.

A treia categorie privește **rezervările și sesiunile de pescuit**. Sistemul trebuie să permită selectarea unui interval, verificarea disponibilității și crearea unei înregistrări persistente. Calculul costului, gestionarea stării rezervării și posibilitatea anulării sunt parte din același flux.

A patra categorie se referă la **pontoane și gestionarea personalului**. Platforma trebuie să permită definirea și administrarea subzonelor rezervabile din interiorul unei locații. Managerul trebuie să poată asocia angajați locației sale.

A cincea categorie acoperă **fluxul de aplicare pentru rolul de manager**. Un utilizator poate propune spre administrare o locație și intra într-un proces controlat de aprobare sau respingere. Administratorul decide, iar decizia sa creează sau nu o locație nouă în platformă.

A șasea categorie reunește **conținutul generat de utilizatori și administrarea contextuală**: recenzii, istoricul repopulărilor cu pești și alte informații care completează experiența utilizatorului și oferă context operațional managerului.

A șaptea categorie vizează **analiza media asistată de inteligență artificială**. Sistemul trebuie să permită încărcarea de imagini și videoclipuri, trimiterea lor spre procesare și afișarea rezultatelor interpretabile. Fluxul este asincron și include validări ale fișierelor,

stări explicite de procesare și prezentarea clară a rezultatelor.

Combinarea acestor cerințe arată că aplicația este simultan un marketplace, un sistem de administrare, o componentă de geolocalizare și un modul de procesare inteligentă. Tocmai această combinație justifică o arhitectură modulară.

3.3 Cerințe non-funcționale și reguli de business

Pe lângă funcționalitățile vizibile, aplicația trebuie să respecte și un set de constrângeri care garantează consistența datelor, securitatea și stabilitatea sistemului.

Securitatea este prima cerință non-funcțională. Platforma trebuie să protejeze identitățile, datele și operațiile sensibile. Controlul accesului este aplicat consecvent la nivel de backend, nu doar la nivel de interfață.

Consistența temporală a rezervărilor este critică. Sistemul trebuie să normalizeze valorile de timp, să valideze că rezervările vizează intervale viitoare și să verifice conflictele cu sesiunile existente. Starea sesiunii (*Pending*, *Confirmed*, *Cancelled*) trebuie urmărită explicit.

Fiabilitatea fluxurilor operaționale presupune că procesele secundare nu blochează fluxurile critice. Dacă trimiterea unui email eșuează, rezervarea nu trebuie anulată. O analiză media cu erori trebuie marcată ca eșuată, fără a afecta restul sistemului.

Performanța și stabilitatea răspunsului impun ca utilizatorul să navigheze rapid și să primească feedback clar pentru operațiile de lungă durată. Procesele costisitoare sunt delegate serviciilor dedicate și executate asincron.

Validarea datelor de intrare este aplicată la nivel de backend. O locație trebuie să existe înainte de a fi rezervată, un ponton trebuie să aparțină locației selectate, recenziile trebuie să respecte un interval valid de rating, iar fișierele media trebuie să respecte restricții de tip și dimensiune.

Urmărirea clară a operațiilor este importantă pentru procese cu impact major: aprobarea unui manager, anularea unei sesiuni sau finalizarea unei analize trebuie să aibă stări și date asociate, urmăribile în timp.

3.4 Proiectarea de ansamblu a sistemului

Pe baza cerințelor formulate, WhereWeFishin este proiectată ca un sistem modular, cu componente separate pe responsabilități. Această alegere răspunde complexității funcționale și nevoii de extindere ulterioară.

Interfața web este orientată spre experiența utilizatorului și spre orchestrarea fluxurilor de interacțiune. Ea filtrează și ghidează acțiunile, dar validările esențiale și deciziile privind accesul sunt tratate în backend. Backend-ul este nucleul logic al sistemului: implementează regulile de autorizare, validările de business, controlul fluxurilor și coordonarea serviciilor externe. Persistența este tratată separat, printr-un strat dedicat și un model relațional centrat pe entitățile domeniului.

Microserviciul de analiză media este separat din motive practice: procesarea imaginilor și videoclipurilor implică dependențe, resurse și timpi de execuție diferiți față de operațiile obișnuite ale aplicației. Izolarea lui reduce cuplarea și limitează impactul operațiilor costisitoare asupra API-ului principal.

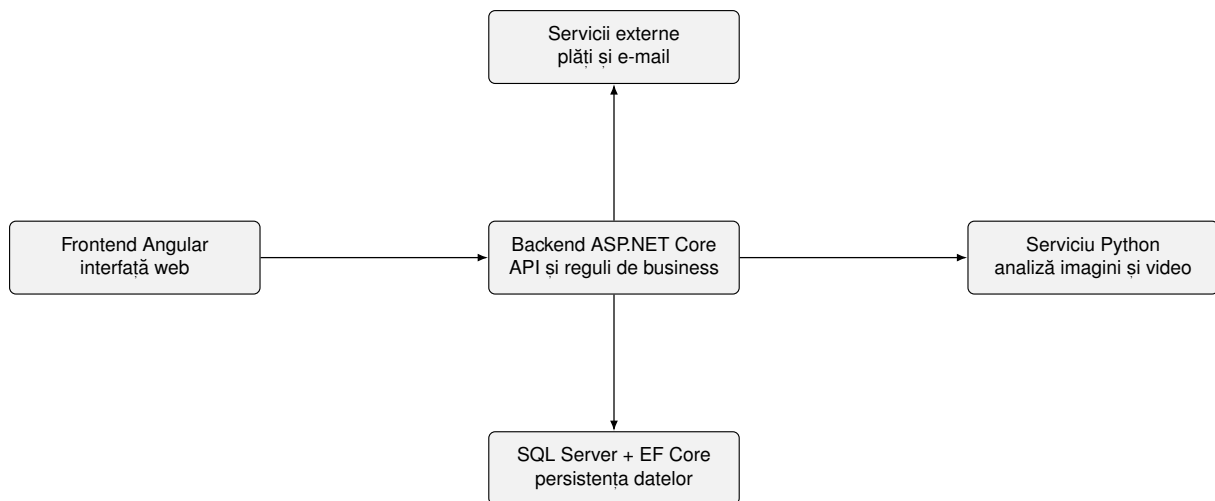


Figura 3.2: Arhitectura de ansamblu a aplicației WhereWeFishin

Serviciile externe auxiliare (procesarea plăților, notificările) sunt integrate astfel încât să poată fi înlocuite fără a afecta structura fluxurilor de business.

3.5 Modelul de domeniu și relațiile dintre entități

Modelul de domeniu descrie obiectele principale ale sistemului și relațiile dintre ele. El trebuie să acopere simultan dimensiunea geospațială, dimensiunea tranzacțională a rezervărilor și dimensiunea operațională a administrării.

Locul de pescuit este entitatea centrală. Reunește informații descriptive, poziționare geografică, tarif, resurse asociate și legături cu actorii care îl administrează. Este punctul de plecare pentru majoritatea fluxurilor importante.

Pontonul este o resursă rezervabilă distinctă, asociată unui loc de pescuit. Introducerea lui răspunde nevoii de delimitare mai precisă a spațiului: utilizatorul nu rezervă întotdeauna întreaga locație, ci adesea o zonă concretă.

Sesiunea de pescuit materializează fluxul de rezervare. Leagă utilizatorul de o locație, de un ponton, de un interval temporal, de un cost și de o stare operațională. Stările explicite (*Pending*, *Confirmed*, *Cancelled*) oferă trasabilitate și susțin regulile de business.

Utilizatorul apare în aproape toate fluxurile. Rolul său nu este unic, ci este diferențiat prin asocierea cu responsabilități diferite. Un utilizator poate iniția rezervări, poate lăsa recenzii sau poate deveni manager printr-un proces formalizat.

Modelul include și **aplicația de manager**, care descrie procesul prin care un utilizator poate ajunge să administreze o locație, și relația **angajat–loc de pescuit**, care permite delegarea controlată a activităților operaționale.

Recenziile, repopulările cu pești și analizele media completează modelul. Analizele media au un ciclu de viață special, cu stări de tip *Pending*, *Processing*, *Completed* și *Failed*, care reflectă natura asincronă a procesării.

Toate aceste entități formează un model coerent, în care obiectele principale sunt conectate prin relații care reflectă activitatea reală a platformei.

Figura 3.3 prezintă o diagramă simplificată a claselor principale, cu atributele cheie ale fiecărei entități.

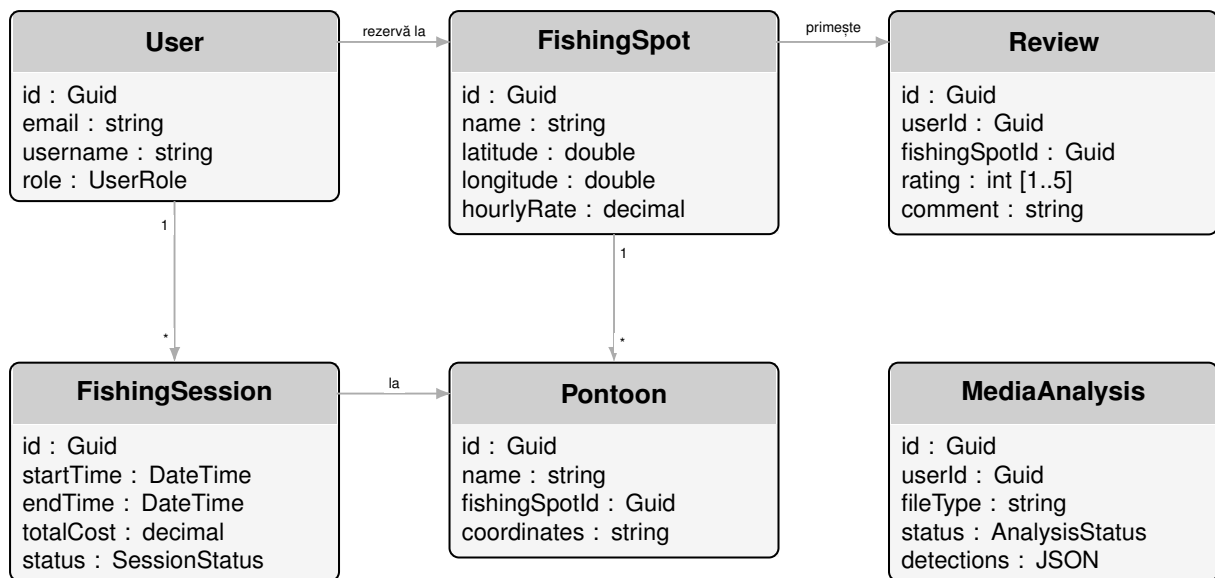


Figura 3.3: Diagrama claselor principale cu atributele entităților de domeniu

3.6 Fluxuri esențiale și diagrame

Figura 3.4 prezintă fluxul principal al unei rezervări, de la alegerea locului de pescuit până la verificarea prin cod QR. Fiecare pas din diagramă corespunde unei operații concrete din sistem: alegerea locului de pescuit implică o interogare filtrată a locațiilor disponibile, selectarea pontonului și intervalului presupune verificarea disponibilității și calculul costului, iar validarea și plata solicită integrarea cu serviciul Stripe. Crearea sesiunii de pescuit este pasul în care toate datele verificate sunt persistate, iar verificarea prin cod QR reprezintă confirmarea fizică a prezenței la locație, mediată de angajat.

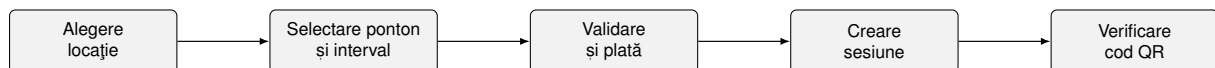


Figura 3.4: Fluxul principal al unei rezervări și al validării sesiunii de pescuit

Al doilea flux important este cel al analizei media, prezentat în figura 3.5. Față de rezervare, care este aproape sincronă, analiza media este un flux asincron, cu stări intermediare vizibile utilizatorului. Încărcarea fișierului declanșează crearea unei înregistrări cu starea *Pending*, iar trimiterea efectivă către serviciul Python inițiază procesarea. Detectarea și urmărirea obiectelor sunt operații costisitoare, a căror durată variază în funcție de dimensiunea fișierului și de complexitatea conținutului. Stocarea rezultatelor marchează finalizarea procesării, iar afișarea în interfață este posibilă imediat după confirmarea stării *Completed*.

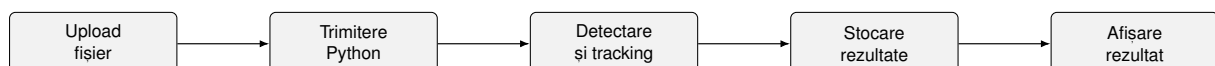


Figura 3.5: Fluxul de analiză media: de la upload până la afișarea rezultatelor

Cele două fluxuri evidențiază o diferență arhitecturală importantă: rezervarea presupune un răspuns relativ rapid (sincron), în timp ce analiza media implică o procesare costisitoare, tratată asincron, cu stări intermediare.

3.7 Sinteza capitolului

Analiza realizată în acest capitol arată că WhereWeFishin este un sistem multi-rol, orientat simultan spre utilizatorul final, administrarea locurilor de pescuit și integrarea unor servicii specializate. Cerințele funcționale acoperă explorarea locațiilor, rezervarea și validarea sesiunilor, administrarea resurselor, procesul de onboarding pentru manageri și procesarea automată a fișierelor media. Cerințele non-funcționale impun constrângeri de securitate, consistență, fiabilitate și trasabilitate.

Pe baza acestor cerințe, sistemul adoptă o arhitectură modulară cu componente separate pentru interfață, logică de business, persistență și analiză media. Modelul de domeniu pune în centru locul de pescuit, sesiunea de pescuit și utilizatorul. Această structură oferă cadrul necesar pentru capitolul următor, în care accentul trece la implementarea efectivă a platformei.

Capitolul 4

Implementare și contribuții personale

Acest capitol descrie cum a fost transpusă în practică soluția prezentată în capitolele anterioare. Dacă în capitolul precedent am discutat cerințele și deciziile de proiectare, acum mă concentrez pe implementarea propriu-zisă a componentelor platformei WhereWeFishin. Scopul nu este prezentarea exhaustivă a codului sursă, ci evidențierea modulelor dezvoltate, alegerilor tehnice și contribuțiilor personale care au făcut posibilă integrarea unui sistem multi-rol, geospațial și asistat de inteligență artificială.

Implementarea este prezentată pe straturi tehnice: mai întâi arhitectura generală, apoi backend-ul, frontend-ul și serviciul de analiză media. În fiecare secțiune am evidențiat explicit deciziile care reflectă contribuția personală.

4.1 Arhitectura implementată a soluției

Implementarea urmează o structură distribuită pe componente cu responsabilități bine delimitate. Interfața este o aplicație Angular, backend-ul este un API HTTP în ASP.NET Core, persistența este gestionată prin Entity Framework Core, iar analiza media este externalizată unui serviciu Python separat. Această separare permite evoluția independentă a modulelor, reduce cuplarea între zone cu cerințe tehnice diferite și menține claritatea codului.

Implementarea urmărește un model cu straturi clare de responsabilitate. La nivelul de transport HTTP, controllerele preiau cererile, aplică autorizarea și delegă logica de business serviciilor. Serviciile orchestrează regulile domeniului, apelează repository-urile pentru acces la date și comunică cu serviciile externe. Repository-urile encapsulează interogările Entity Framework Core, menținând separarea dintre logica domeniului și detaliile de persistență. Această structură reduce cuplarea și permite testarea independentă a fiecărui strat.

Configurarea aplicației ASP.NET Core urmează un lanț de middleware definit explicit: autentificare, autorizare, compresie, CORS, rate limiting și rutare. Ordinea middleware-ului este importantă deoarece determină când se aplică fiecare politică și garantează că cererile neautorizate sunt respinse înainte de a atinge logica de business. Injecția de dependențe este utilizată sistematic: fiecare serviciu este înregistrat cu durată de viață corespunzătoare (tranzient, scoped sau singleton) în funcție de natura operației.

Contribuția personală este vizibilă încă de la nivelul arhitecturii: am ales să separ operațiile tranzacționale uzuale de prelucrarea media costisitoare. Față de o abordare monolitică, comunicarea explicită dintre frontend, API și serviciul de analiză a simplificat

atât dezvoltarea incrementală, cât și depanarea fluxurilor complexe.

4.2 Implementarea backend-ului

Backend-ul este nucleul logic al aplicației și zona în care sunt concentrate majoritatea regulilor de business. Implementarea sa se bazează pe controllere specializate, servicii reutilizabile și un strat de persistență bazat pe repository-uri. Această structură separă logica de transport HTTP de logica domeniului și de accesul la date, reducând duplicarea și simplificând testarea.

Configurația generală a aplicației definește autentificarea JWT, limitarea dimensiunii cererilor, compresia răspunsurilor, politicile CORS, rate limiting-ul pentru endpoint-urile sensibile și clienții HTTP necesari integrării cu serviciul Python. Această zonă arată că backend-ul nu tratează doar operații CRUD, ci gestionează și securitatea, performanța și interoperabilitatea.

4.2.1 Autentificare, autorizare și controlul accesului

Modulul de autentificare separă clar rolurile și protejează operațiile sensibile. Autentificarea se bazează pe tokenuri JWT, iar accesul la resurse este controlat prin claims și reguli de autorizare reutilizabile. Această soluție menține un model coerent pentru toate tipurile de utilizatori și reduce cuplarea dintre controllere și detaliile privind identitatea curentă.

Endpoint-urile de autentificare gestionează înregistrarea, autentificarea, verificarea tokenului și recuperarea accesului. Rata cererilor este limitată pentru a reduce riscul de abuz. Informațiile despre utilizator sunt extrase centralizat din claims, evitând repetarea logicii în fiecare controller.

Am introdus extensii de autorizare pentru resursele dependente de un loc de pescuit, astfel încât verificarea dreptului de administrare să poată fi refolosită în module diferite: pontoane, angajați, repopulări. Aceasta a permis extinderea aplicației cu noi module fără a duplica logica de acces.

Figura 4.1 ilustrează fluxul de autentificare, de la cererea clientului până la validarea tokenului JWT.



Figura 4.1: Fluxul de autentificare bazat pe tokenuri JWT

4.2.2 Locuri de pescuit, pontoane și organizarea resurselor

Implementarea locurilor de pescuit este unul dintre modulele centrale ale backend-ului, deoarece aproape toate fluxurile importante pornesc din această entitate. Controlerul gestionează listarea, filtrarea și administrarea locațiilor, cu interogări diferențiate în funcție de rolul utilizatorului. Am mutat filtrarea autorizată cât mai aproape de nivelul interogării, astfel încât datele să nu fie încărcate integral și filtrate doar la nivel de răspuns.

Pentru pontoane am implementat un modul separat care asociază zone distincte unui loc de pescuit și menține o legătură directă între componenta geospațială din interfață și regulile de disponibilitate din backend. Pontonul devine astfel o resursă rezervabilă explicită, nu doar o informație descriptivă. Această abordare este importantă pentru scenarii reale în care o locație conține mai multe spații cu disponibilitate independentă.

Coordonatele geografice ale pontoanelor sunt stocate ca liste de puncte ce descriu conturul poligonal al zonei rezervabile. Această alegere permite reprezentarea fidelă a formei unui ponton pe hartă, nu doar a centrului său. La adăugarea sau modificarea unui ponton din interfața managerului, datele geospațiale sunt serializate în format GeoJSON, transmise la backend și stocate în baza de date. La listare, coordonatele sunt deserializate și trimise frontend-ului, care le interpretează direct ca layer Leaflet. Această soluție simplifică persistența fără a necesita o extensie geospațială dedicată a bazei de date, păstrând compatibilitatea cu operațiile de afișare și modificare din interfață.

Contribuția personală constă în modelarea coerentă a relației dintre entitatea principală și subzonele rezervabile, precum și în reutilizarea aceluiași reguli de autorizare pentru toate operațiile de administrare.

4.2.3 Workflow-ul de aplicare pentru manager

Unul dintre cele mai interesante module pe care le-am implementat este fluxul de aplicare pentru rolul de manager. În locul creării directe a unui loc de pescuit, aplicația introduce un proces controlat: utilizatorul transmite datele locației propuse, iar administratorul decide aprobarea sau respingerea cererii.

Am definit explicit stările aplicației, am păstrat istoricul de evaluare și am asociat clar decizia administrativă de crearea resursei în sistem. Controllerul gestionează crearea cererii, actualizarea ei în condiții controlate, retransmiterea după respingere și aprobarea finală. Am limitat tranzițiile de stare pentru a evita inconsistențe: nu se poate edita o cerere deja aprobată și nu pot exista două cereri active pentru același utilizator.

Entitatea aplicației păstrează informații despre administratorul evaluator, momentul evaluării și motivul respingerii. Această transparentă face fluxul ușor de urmărit.

Această zonă reprezintă o contribuție personală importantă deoarece combină modelul de domeniu, regulile de business și administrarea controlată a creșterii platformei.

4.2.4 Rezervări, sesiuni de pescuit și integrarea plății

Fluxul de rezervare este cel mai dens din punct de vedere al logicii de business. La nivel de implementare, sistemul nu creează pur și simplu o înregistrare, ci validează locul de pescuit, verifică disponibilitatea pentru intervalul ales, corelează opțional un ponton, calculează costul și tratează separat integrarea plății.

Legătura cu Stripe este separată clar de persistența sesiunii finale. Am generat un *PaymentIntent* care păstrează metadata relevante despre utilizator, locație și interval. Această separare oferă o bază mai sigură pentru corelarea plății cu sesiunea de pescuit și pentru evitarea erorilor de sincronizare. Modelul intern al sesiunii normalizează temporal datele și păstrează o stare explicită: *Pending*, *Confirmed* sau *Cancelled*.

Detectarea conflictelor temporale este implementată printr-o interogare LINQ evaluată direct la nivel de bază de date. Condiția de conflict pentru un ponton dat verifică dacă există sesiuni confirmate sau în așteptare ale căror intervale se intersectează cu

cel solicitat, adică sesiuni care încep înainte de finalul intervalului nou și se termină după începutul acestuia. Această formulare acoperă toate cazurile de suprapunere parțială sau totală și evită încărcarea inutilă a datelor în memorie. Rezultatul este un mesaj explicit de conflict transmis clientului, care include intervalul neeligibil, astfel încât interfața poate evidenția perioada ocupată fără a necesita interogări suplimentare.

Contribuția personală în acest modul este integrarea coerentă a regulilor de disponibilitate, a modelului de sesiune și a unei plăți externe, fără a pierde consistența internă a datelor.

4.2.5 Angajați, recenzii și servicii auxiliare

Backend-ul include și componente care completează partea operațională. Modulul de angajați permite asocierea utilizatorilor la locuri de pescuit, creând o delegare controlată a operațiilor fără ridicarea privilegiilor la nivel de manager. Relația angajat–locăție este gestionată explicit, cu verificarea că utilizatorul asociat există și că nu deține deja un rol mai ridicat pentru locația respectivă, prevenind suprapunerile de responsabilitate.

Sistemul de recenzii introduce feedback din partea utilizatorilor pentru locurile de pescuit vizitate. Recenziile sunt validate la nivel de rating și lungime a textului, iar interogările care le agregă pot fi cache-uite la nivel de răspuns HTTP pentru a reduce încărcarea bazei de date în scenarii cu trafic ridicat. Invalidarea cache-ului se face explicit la adăugarea sau modificarea unei recenzii, asigurând că datele prezentate sunt actuale.

Serviciul de notificări prin email este integrat la momentele cheie ale fluxurilor: confirmarea rezervării, aprobarea sau respingerea aplicației de manager și notificările despre analizele media finalizate. Implementarea folosește un serviciu asincron cu comportament de tip fire-and-forget: dacă trimiterea eșuează, operația principală nu este afectată și nu este anulată. Această izolare este importantă pentru ca un furnizor de email indisponibil temporar să nu blocheze rezervările sau alte fluxuri critice.

Contribuția personală constă în transformarea unor funcții aparent secundare într-o infrastructură stabilă care îmbunătățește experiența utilizatorului fără a compromite fiabilitatea sistemului.

4.3 Implementarea frontend-ului

Frontend-ul WhereWeFishin este o aplicație Angular modernă, bazată pe componente standalone, routing cu încărcare lazy și guard-uri funcționale. Această alegere a permis organizarea interfeței în zone funcționale clare și separarea fluxurilor destinate utilizatorului obișnuit, managerului, angajatului și administratorului.

4.3.1 Structura aplicației SPA și controlul accesului în interfață

La baza implementării se află structura traseelor și guard-urile funcționale. Fiecare zonă importantă se încarcă lazy și este protejată în funcție de rolul utilizatorului. Guard-urile funcționale verifică atât prezența tokenului JWT, cât și rolul din claims, redirectionând utilizatorii neautorizați înainte ca modulul să fie descărcat. Configurarea interceptorilor HTTP permite atașarea automată a tokenului la fiecare cerere și tratarea

controlată a erorilor recurente, precum expirarea autentificării: în acest caz, utilizatorul este deconectat și redirecționat spre pagina de autentificare.

Structura de module standalone a facilitat și separarea configurației de rutare: zona utilizatorului, zona managerului, zona angajatului și zona administratorului sunt module independente, cu propriile rute și guard-uri, ceea ce simplifică atât întreținerea, cât și extinderea ulterioară cu noi zone funcționale.

Contribuția personală constă în distribuirea coerentă a logicii de acces între rutare, guard-uri și interceptoare, astfel încât comportamentul interfeței să rămână aliniat permanent cu regulile de securitate aplicate de API.

4.3.2 Harta interactivă și explorarea locurilor de pescuit

Una dintre cele mai importante componente frontend este pagina de explorare, construită în jurul hărții interactive. Am combinat date despre locații, geolocalizarea utilizatorului, rutare pe hartă și informații despre specii. Am dezvoltat helperi dedicați pentru markere personalizate, stilizarea elementelor geografice și construirea ferestrelor informative într-un mod reutilizabil.

Sistemul de markere personalizate utilizează template-uri SVG generate dinamic, colorate diferit în funcție de disponibilitate sau tipul de marker. Ferestrele informative sunt construite din template-uri HTML compilate separat și injectate în Leaflet ca stringuri, o abordare care păstrează compatibilitatea cu biblioteca de hartă fără a introduce componente Angular în afara contextului normal de rendering. Helperii dedicați pentru tooltip-uri și badge-uri permit reutilizarea aceluiași cod de generare a conținutului vizual în mai multe contexte: harta principală de explorare, harta de administrare a managerului și componentele de detaliu ale locației.

Harta funcționează ca punct de intrare în platformă: utilizatorul identifică rapid locațiile disponibile, vede proximitatea față de poziția sa și decide ce loc merită explorat mai departe. Contribuția personală constă în integrarea componentei Leaflet cu datele reale ale aplicației și în transformarea hărții dintr-un element vizual într-un instrument central de navigare.

4.3.3 Coșul, calendarul și integrarea Stripe în interfață

Fluxul de rezervare are o componentă frontend complexă, cu mai multe stări și validări vizuale. Utilizatorul selectează intervale orar, vede zilele ocupate, poate combina mai multe elemente în coș și finalizează plata. Calendarul pe care l-am implementat evidențiază perioadele rezervate cu granularitate de jumătate de zi, tratează corect intervalele parțiale, în care pontonul este disponibil doar pentru o parte din zi, și blochează selecțiile invalide înainte ca utilizatorul să ajungă la pasul de plată. Această validare vizuală reduce numărul de erori raportate de API și îmbunătățește experiența de rezervare.

Integrarea Stripe se bazează pe Stripe Elements, care încarcă dinamic câmpurile de plată direct în pagină, fără redirecționare externă. Logica de confirmare a plății este separată de logica de afișare: componenta frontend așteaptă confirmarea *PaymentIntent* de la Stripe înainte de a solicita backend-ului crearea sesiunii de pescuit, evitând situațiile în care o sesiune este creată fără ca plata să fie finalizată.

Contribuția personală constă în rezolvarea unor probleme practice de UX și sincronizare: vizualizarea clară a indisponibilităților, evitarea selecțiilor greșite și corelarea

coerentă a stărilor de plată cu răspunsurile backend-ului.

4.3.4 Wizard-ul de aplicare și dashboard-ul operațional

Zona de manager include două componente cu valoare tehnică ridicată. Wizard-ul de aplicare este organizat pe mai mulți pași și combină date descriptive, alegerea locației pe hartă și o revizuire finală înainte de trimitere. Fiecare pas validează datele introduse înainte de a permite avansarea, astfel încât erorile sunt interceptate timpuriu, nu la trimiterea finală a formularului. Starea completată a fiecărui pas este păstrată în memorie pe durata sesiunii, permițând utilizatorului să revină la pași anteriori fără a pierde datele introduse.

Dashboard-ul managerului reunește administrarea pontoanelor, gestionarea angajaților, actualizarea datelor locației și vizualizarea informațiilor operaționale. Componentele sunt încărcate independent, astfel că o secțiune cu date lente, de exemplu lista rezervărilor, nu blochează afișarea celorlalte. Interacțiunile cu harta pentru administrarea pontoanelor sunt gestionate printr-un serviciu dedicat care asigură sincronizarea între starea grafică a hărții și modelul de date persistent.

Dashboard-ul este relevant deoarece concentrează mai multe subfluxuri distincte într-o interfață unitară coerentă. Contribuția personală constă în construirea unei interfețe capabile să deservească activități operaționale reale, cu stări clare și feedback imediat la fiecare acțiune.

4.3.5 Verificarea prin cod QR și funcțiile administrative

Am implementat un flux de verificare a sesiunilor prin cod QR conceput pentru utilizare în teren. Componenta accesează camera dispozitivului prin API-ul browser-ului, procesează în timp real imaginea captată și extrage datele codului. Tratarea datelor rezervării este suficient de tolerantă pentru a gestiona variații ale structurii lor, de exemplu coduri generate de versiuni anterioare ale aplicației sau cu formatare ușor diferită, fără a respinge coduri valide din motive formale. Odată decodat codul, componenta interoghează API-ul pentru detaliile sesiunii și prezintă un rezumat clar angajatului, cu opțiunea de confirmare.

Zona administrativă oferă funcții de supraveghere asupra utilizatorilor înregistrați, locațiilor active și aplicațiilor de manager în așteptare. Am implementat componente tabelare cu filtrare și paginare, astfel încât listele să rămână utilizabile chiar și atunci când numărul de înregistrări crește. Fluxul de aprobare a aplicațiilor include stări vizuale clare pentru fiecare cerere și acțiuni rapide de decizie: aprobare sau respingere cu motivație.

Integrarea tuturor tipurilor de fluxuri într-o singură aplicație SPA a necesitat o separare clară a permisiunilor și a traseelor, asigurând coerența navigării și securitatea accesului la date.

4.4 Implementarea sistemului de analiză media

Sistemul de analiză media este componenta care diferențiază cel mai clar WhereWeFishin de o aplicație web convențională de rezervări. Implementarea implică

colaborarea dintre trei straturi: interfața Angular, backend-ul .NET care orchestrează fluxul și serviciul Python care execută analiza.

Spre deosebire de celelalte fluxuri din platformă, cum ar fi rezervări, recenzii sau administrare, analiza media nu poate fi tratată sincron. Un fișier video poate necesita câteva zeci de secunde sau chiar minute pentru a fi procesat, iar blocarea unui thread HTTP pe toată această durată ar face sistemul nefuncțional sub orice trafic real. De aceea, componenta a fost proiectată în jurul unui model asincron explicit, cu un ciclu de viață format din patru stări persistate: *Pending*, *Processing*, *Completed* și *Failed*. Aceste stări sunt accesibile în orice moment prin polling din frontend și recuperabile după un restart al serverului.

Coordonarea celor trei straturi urmează un protocol clar: utilizatorul încarcă fișierul prin interfața Angular, care îl trimite la backend-ul .NET printr-o cerere multipart. Backend-ul validează fișierul, creează înregistrarea de analiză în starea *Pending* și returnează imediat un identificator unic clientului, fără a aștepta finalizarea procesării. Trimiterea efectivă la serviciul Python este inițiată asincron: starea analizei devine *Processing* pe durata apelului, iar la primirea rezultatelor trece în *Completed*. Dacă serviciul Python este indisponibil sau returnează o eroare, analiza este marcată ca *Failed* fără a propaga excepția spre utilizator.

```
1 public async Task<MediaAnalysisResult> SendToAnalysisServiceAsync(  
2     Stream fileStream, string fileName, string mimeType,  
3     CancellationToken ct = default)  
4 {  
5     using var content = new MultipartFormDataContent();  
6     var fileContent = new StreamContent(fileStream);  
7     fileContent.Headers.ContentType =  
8         new MediaTypeHeaderValue(mimeType);  
9     content.Add(fileContent, "file", fileName);  
10  
11     var response = await _httpClient.PostAsync(  
12         "/analyze", content, ct);  
13     response.EnsureSuccessStatusCode();  
14  
15     return await response.Content  
16         .ReadFromJsonAsync<MediaAnalysisResult>(ct)  
17         ?? throw new InvalidOperationException(  
18             "Empty response from analysis service");  
19 }
```

Listing 4.1: Trimiterea fișierului media la serviciul Python și actualizarea stării

Clientul HTTP care comunică cu serviciul Python este configurat cu un timeout extins, necesar pentru videoclipurile lungi, și cu politici de retry limitate la erorile tranziente de rețea. Separarea serviciului HTTP de controllerul de upload permite testarea independentă: controllerul poate fi testat cu un serviciu simulat care returnează rezultate predefinite, iar serviciul HTTP poate fi testat izolat față de logica de persistență. Validarea fișierelor înaintea trimiterii include verificarea tipului MIME față de o listă albă și limitarea dimensiunii: imagini până la 10 MB, videoclipuri până la 150 MB. Fișierele care nu respectă aceste constrângeri sunt respinse la nivelul controller-ului, fără a atinge serviciul Python, reducând astfel riscul procesării unor fișiere corupte sau cu format neașteptat.

4.4.1 Integrarea dintre backend și serviciul Python

La nivel de backend, un serviciu dedicat gestionează comunicarea cu aplicația Python: trimite fișierele spre procesare, interpretează răspunsul primit și actualizează entitățile persistente. Comunicarea se face prin HTTP, cu timeout configurabil și mecanism de retry pentru cazurile în care serviciul Python nu răspunde imediat. Dacă serviciul este indisponibil, analiza este marcată ca *Failed*, fără a afecta restul aplicației.

Controllerele expun endpoint-uri pentru încărcare, listare, detalieri și ștergere a analizelor, cu reguli clare de autorizare: utilizatorul poate accesa doar propriile analize, iar managerul poate consulta analizele asociate locației sale. Validarea fișierelor la upload include verificarea tipului MIME și a dimensiunii, cu limite configurate explicit pentru imagini și videoclipuri.

Am introdus stări explicite pentru fiecare analiză: *Pending* (înregistrată, neîncepută), *Processing* (trimisă spre serviciul Python), *Completed* (rezultate disponibile) și *Failed* (procesare eșuată). Aceste stări permit urmărirea ciclului de viață al procesării, oferă frontend-ului o bază clară pentru polling și fac auditul erorilor de procesare posibil fără a necesita loguri externe.

Contribuția personală constă în transformarea unui apel extern către un serviciu specializat de viziune artificială într-un flux controlat, persistent și ușor de monitorizat.

4.4.2 Pipeline-ul video: detecție, tracking și post-procesare

În serviciul Python, analiza video pornește de la încărcarea modelului YOLO și configurarea tracker-ului ByteTrack cu parametrii adaptați domeniului: praguri de încredere pentru detecție, dimensiunea minimă a bounding box-ului și numărul de cadre de toleranță pentru reidentificarea unui obiect pierdut temporar. Fiecare cadru video este analizat de modelul YOLO pentru detectarea obiectelor relevante, iar ByteTrack asociază detecțiile succesive aceluiași identificator de obiect între cadre, evitând astfel contorizarea multiplă a aceluiași exemplar.

Traectoriile vizuale ale obiectelor sunt păstrate ca trasee temporale, cu momentele de apariție și dispariție pentru fiecare identificator urmărit. Această informație permite generarea unui rezumat statistic complet: numărul total de exemplare distincte, speciile dominante și distribuția temporală a detecțiilor. Rezultatul video este reîncodat cu FFmpeg pentru a produce un fișier cu adnotările vizibile suprapuse, care poate fi servit direct în interfață.

Contribuția personală se reflectă în calibrarea pragurilor, alegerea mecanismului de vizualizare a traseelor și proiectarea formei în care rezultatele brute ale modelului sunt convertite în structuri stocabile și ușor de interpretat de frontend.

4.4.3 Afișarea rezultatelor în frontend și tratarea stării asincrone

În frontend, modulele pentru recunoaștere video și clasificare pe imagini gestionează nu doar upload-ul fișierelor, ci și o stare asincronă care evoluează în timp. Imediat după încărcarea unui fișier, componenta afișează starea *Pending* și pornește un mecanism de polling cu interval configurabil: la fiecare câteva secunde, se interogă backend-ul pentru actualizarea stării analizei. Odată ce starea devine *Completed* sau *Failed*, polling-ul se oprește și interfața tranzitionează automat la afișarea rezultatelor sau a mesajului de eroare.

Afișarea rezultatelor pentru imagini include vizualizarea bounding box-urilor detectate suprapuse pe imagine originală, o listă a speciilor identificate cu gradele de încredere asociate și statistici agregate. Pentru video, interfața oferă playerul cu adnotările vizibile, rezumatul temporal al detecțiilor și istoricul complet al exemplarelor urmărite pe durata înregistrării.

Optimizările pentru zona video includ încărcarea întârziată a elementelor media grele, paginarea listelor de rezultate și comprimarea răspunsurilor de la backend. Acestea reduc consumul de date și mențin interfața responsivă chiar și pentru videoclipuri cu multe detecții.

Contribuția personală constă în conectarea coerentă a stărilor persistente din backend cu mecanismele de feedback vizual din interfață, construind un flux complet care gestionează așteptarea, erorile de procesare și prezentarea rezultatelor într-o formă utilă utilizatorului.

4.5 Infrastructura de deployment

Aplicația WhereWeFishin rulează în producție pe o mașină locală, expusă public prin Cloudflare Tunnel, fără a necesita o adresă IP statică sau configurarea manuală a porturilor în router. Această abordare combină simplitatea operațională cu un nivel ridicat de securitate: tot traficul trece printr-o conexiune criptată spre infrastructura Cloudflare, mașina gazdă nu expune niciun port public, iar certificatele TLS sunt gestionate automat. Componentele aplicației sunt containerizate cu Docker și orchestrate cu Docker Compose, asigurând portabilitate și consistență între medii.

4.5.1 Containerizarea cu Docker și build-uri multi-etapă

Fiecare componentă a aplicației rulează într-un container Docker separat [8], orchestrate împreună cu Docker Compose. Structura cuprinde patru servicii principale: baza de date SQL Server, backend-ul ASP.NET Core, serviciul Python de analiză media și frontend-ul Angular servit prin Nginx.

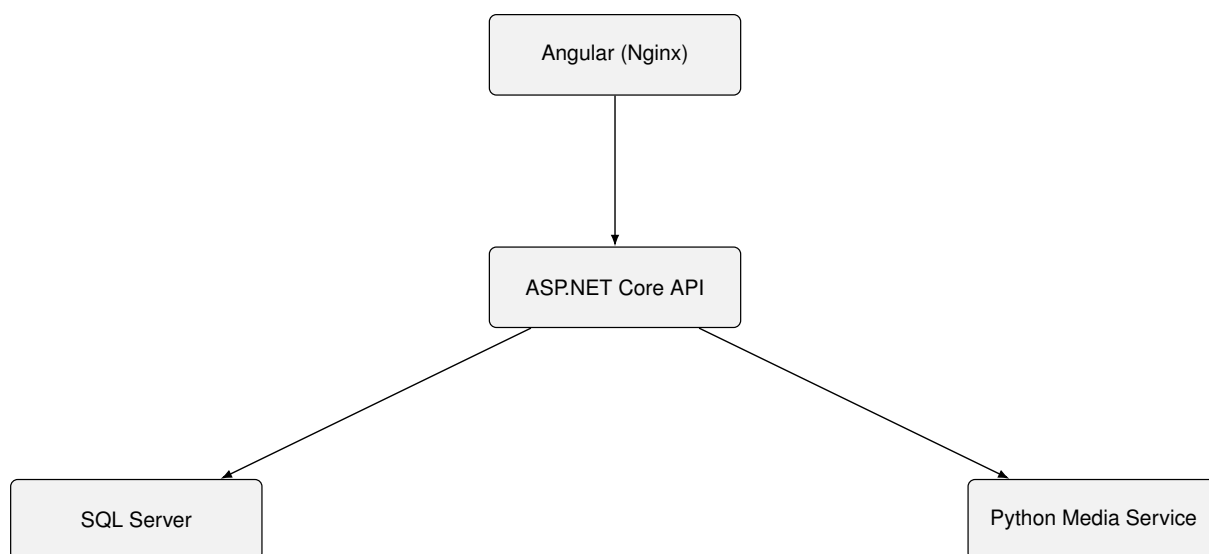


Figura 4.2: Serviciile Docker și dependențele dintre ele

Imaginile Docker sunt construite prin *Dockerfile*-uri cu build multi-etapă. Separarea etapei de compilare de cea de execuție reduce semnificativ dimensiunea imaginii finale și elimină din containerul de producție toate uneltele de build, cum ar fi compilatorul .NET SDK, sursele și fișierele intermediare, care nu sunt necesare la rulare și ar reprezenta o suprafață de atac suplimentară.

```
1 # Etapa 1: compilare
2 FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
3 WORKDIR /src
4 COPY ["WhereWeFishin.API/WhereWeFishin.API.csproj",
5      "WhereWeFishin.API/"]
6 RUN dotnet restore "WhereWeFishin.API/WhereWeFishin.API.csproj"
7 COPY . .
8 WORKDIR "/src/WhereWeFishin.API"
9 RUN dotnet publish -c Release -o /app/publish
10
11 # Etapa 2: runtime (imagine mai mica, fara SDK)
12 FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
13 WORKDIR /app
14 COPY --from=build /app/publish .
15 ENTRYPOINT ["dotnet", "WhereWeFishin.API.dll"]
```

Listing 4.2: Dockerfile multi-etapă pentru backend-ul ASP.NET Core

Același principiu se aplică pentru frontend: Angular CLI compilează aplicația în etapa de build, iar imaginea finală conține exclusiv fișierele statice servite de Nginx. Fișierul `docker-compose.yml` definește rețeaua internă comună tuturor serviciilor, volumele persistente pentru baza de date și fișierele media, precum și ordinea de pornire prin directiva `depends_on` cu condiție de tip `service_healthy`. Backend-ul nu pornește până când SQL Server nu trece health check-ul, prevenind erori de conexiune la pornire.

Modelul antrenat YOLOv8 este montat ca volum read-only în containerul Python (`:ro`), fără a fi inclus în imagine. Această decizie permite actualizarea modelului fără a reconstrui sau republica containerul. Variabilele de mediu sensibile, cum ar fi șirul de conexiune la baza de date, secretul JWT și cheile Stripe, sunt injectate prin fișierul `.env` exclus din controlul versiunilor, separând configurația de codul sursă.

4.5.2 Rate limiting și securitatea pipeline-ului HTTP

Pipeline-ul HTTP al backend-ului include mai multe straturi de protecție configurate explicit în `Program.cs`. Rate limiting-ul este primul mecanism de apărare pentru endpoint-urile expuse la abuz: ASP.NET Core 7+ include middleware dedicat pentru limitarea numărului de cereri per interval de timp, fără dependențe externe.

```
1 builder.Services.AddRateLimiter(options =>
2 {
3     // Politica stricta pentru autentificare
4     options.AddFixedWindowLimiter("auth", policy =>
5     {
6         policy.PermitLimit    = 5;
7         policy.Window         = TimeSpan.FromMinutes(1);
8         policy.QueueLimit     = 0;
9     });
10
11     // Politica pentru upload media
12     options.AddFixedWindowLimiter("upload", policy =>
```

```

13     {
14         policy.PermitLimit    = 10;
15         policy.Window         = TimeSpan.FromMinutes(1);
16         policy.QueueLimit     = 2;
17     });
18
19     options.RejectionStatusCode = StatusCodes.
        Status429TooManyRequests;
20 });

```

Listing 4.3: Configurarea rate limiting-ului cu politici diferențiate

Endpoint-urile de autentificare aplică politica "auth", configurată prin atributul `EnableRateLimiting` pe acțiunile de login: un client care depășește 5 cereri pe minut primește HTTP 429, prevenind atacurile de tip brute-force. Endpoint-urile de upload folosesc politica "upload", cu o coadă de 2 cereri pentru a absorbi vârfuri scurte fără a respinge imediat utilizatorii legitimi.

Compresia răspunsurilor este configurată prin middleware-ul `ResponseCompression`, cu suport pentru Brotli și Gzip, activat condiționat în funcție de header-ul `Accept-Encoding` al clientului. Aceasta reduce lățimea de bandă consumată de răspunsurile JSON voluminoase, în special listele de locații cu metadata completă și rezultatele analizelor media cu multe detecții.

Politicile CORS sunt definite separat pentru mediul de dezvoltare și producție. În dezvoltare, originile sunt permise larg pentru a facilita lucrul local. În producție, politica restricționează `Origin` la domeniile aplicației, blocând cererile încrucișate de la surse neautorizate:

```

1  builder.Services.AddCors(options =>
2  {
3      options.AddPolicy("Production", policy =>
4          policy.WithOrigins(
5              "https://wherefishin.ro",
6              "https://www.wherefishin.ro")
7              .AllowAnyHeader()
8              .AllowAnyMethod()
9              .AllowCredentials());
10
11      options.AddPolicy("Development", policy =>
12          policy.AllowAnyOrigin()
13              .AllowAnyHeader()
14              .AllowAnyMethod());
15  });
16
17  var corsPolicy = app.Environment.IsProduction()
18      ? "Production" : "Development";
19  app.UseCors(corsPolicy);

```

Listing 4.4: Politica CORS diferențiată pe mediu de execuție

Limitarea dimensiunii corpului cererilor HTTP este configurată la nivelul serverului Kestrel: cererile obișnuite au o limită de 5 MB, iar endpoint-urile de upload media sunt configurate explicit la 150 MB. Fișierele care depășesc această limită sunt respinse de Kestrel înainte de a atinge controllerul, fără a consuma resurse de procesare.

4.5.3 Health check-uri și reziliența serviciilor

Fiecare serviciu Docker expune un health check configurat în `docker-compose.yml`. Health check-ul SQL Server verifică că motorul de baze de date acceptă conexiuni prin execuția unui `SELECT 1` cu credențialele de serviciu. ASP.NET Core expune un endpoint `/health` prin middleware-ul `HealthChecks`, care verifică conexiunea la baza de date și disponibilitatea serviciului Python.

Această configurare servește două scopuri. La pornire, Docker Compose folosește starea health check-urilor pentru a ordona inițializarea serviciilor: backend-ul nu pornește până când baza de date nu este gata. În producție, un orchestrator sau un script de monitorizare poate detecta și reporni automat serviciile degradate. Separarea health check-ului de logica aplicației menține un punct de verificare neutral, disponibil fără autentificare, dar inaccesibil public prin configurația Cloudflare Tunnel.

Reziliența comunicării dintre backend și serviciul Python este implementată prin politici de retry configurate pe clientul HTTP. Erorile tranziente de rețea, cum ar fi timeout sau conexiune refuzată, declanșează reîncercarea de maximum două ori, cu interval exponențial. Erorile permanente (HTTP 4xx de la serviciul Python) nu sunt reîncercate, deoarece retrimiteria aceluiasi fișier invalid nu va produce un rezultat diferit. Această separare între erori tranziente și permanente reduce zgomotul din loguri și evită supraîncărcarea unui serviciu deja degradat.

4.5.4 Expunerea publică prin Cloudflare Tunnel

Cloudflare Tunnel (`cloudflared`) rulează ca un serviciu suplimentar în stiva Docker Compose. La pornire, deschide o conexiune persistentă criptată spre infrastructura Cloudflare și înregistrează regulile de rutare configurate în panoul Cloudflare Zero Trust. Traficul de la utilizatori ajunge la serverele Cloudflare, care îl redirecționează prin tunel spre containerele locale, fără ca nicio adresă IP sau port să fie expus direct pe internet. Această topologie elimină o categorie întreagă de riscuri: scanarea porturilor, atacurile directe asupra IP-ului mașinii și exploatarea unor servicii expuse accidental.

Configurația tunelului asociază subdomeniile aplicației serviciilor interne prin numele lor Docker Compose. Cererea spre `api.wherfishin.ro` este trimisă la `http://api:8080`, iar cererea spre `wherfishin.ro` la `http://frontend:80`. Serviciul Python nu apare în configurația de rutare și rămâne accesibil exclusiv din rețeaua internă Docker.

Cloudflare gestionează automat certificatele TLS și reînnoirea lor. Prin intermediul Cloudflare WAF (Web Application Firewall), traficul este filtrat și înainte de a ajunge la tunel: regulile de tip OWASP sunt aplicate implicit, iar adresele IP cu comportament abuziv pot fi blocate la nivelul rețelei Cloudflare, fără a atinge mașina gazdă. Protecția DDoS funcționează prin absorbția traficului în infrastructura distribuită Cloudflare, nu prin capacitatea mașinii locale.

Contribuția personală în această zonă constă în configurarea completă a stivei Docker Compose, scrierea *Dockerfile*-urilor optimizate pentru fiecare serviciu, definirea politicilor de rate limiting și CORS diferențiate pe mediu și integrarea Cloudflare Tunnel ca mecanism de expunere securizată, fără a compromite izolarea rețelei interne.

4.6 Contribuțiile principale în perspectivă de ansamblu

Proiectul WhereWeFishin reunește contribuții personale semnificative în cinci zone funcționale distincte, fiecare cu propriile provocări tehnice și decizii de implementare. Această distribuție nu a fost planificată a priori, ci a rezultat din necesitățile reale ale platformei: un sistem multi-rol, geospațial și asistat de inteligență artificială nu poate fi construit dintr-un singur centru de greutate tehnic.

Componenta de **rezervări și sesiuni** este cea mai densă din perspectiva regulilor de business: validarea disponibilității, calculul costului, corelarea plății Stripe cu starea sesiunii și gestionarea conflictelor temporale. Componenta de **administrare a locațiilor** acoperă workflow-ul de aplicare pentru manager, pontoanele cu geometrie geospațială și relația angajat–locație. Componenta de **analiză media AI** reprezintă integrarea microserviciului Python cu pipeline-ul de detecție YOLO și tracking ByteTrack, plus ciclul de viață asincron gestionat între backend și frontend. Componenta de **interfață și experiență utilizator** include harta interactivă Leaflet, calendarul de disponibilitate, wizard-ul de aplicare și fluxul de plată Stripe Elements. Modulul de **autentificare și securitate** asigură separarea rolurilor, autorizarea bazată pe claims și protecția endpoint-urilor sensibile.

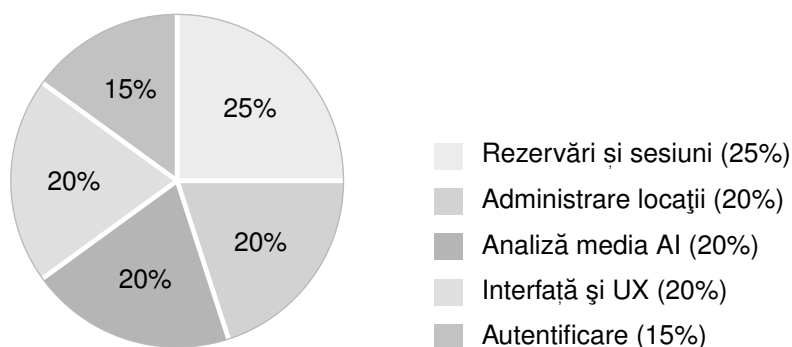


Figura 4.3: Distribuția contribuțiilor personale pe zone funcționale ale platformei

Proporțiile din figura 4.3 reflectă complexitatea relativă a fiecărei zone, estimată prin numărul de componente implementate, densitatea regulilor de business și gradul de integrare cu servicii externe. Nu există o zonă neglijabilă: autentificarea, deși cea mai mică ca pondere, este fundația pe care se sprijină controlul accesului în toate celelalte module.

4.7 Sinteza capitolului

Implementarea WhereWeFishin arată cum o soluție analizată la nivel de cerințe și proiectare poate fi transpusă într-un sistem coerent, multi-strat și orientat spre utilizare reală. Backend-ul concentrează regulile de business și integrarea serviciilor externe, frontend-ul construiește experiența utilizatorului și orchestrează fluxurile operaționale, iar serviciul de analiză media extinde aplicația cu o componentă de viziune artificială integrată controlat.

Contribuția personală se regăsește în toate aceste zone: în proiectarea și implementarea workflow-ului pentru manageri, în formalizarea rezervărilor și a plăților, în construirea unei interfețe geospațiale consistente și în integrarea unui pipeline AI capabil

să proceseze atât imagini, cât și videoclipuri. Infrastructura de deployment asigură că toate aceste componente pot fi pornite și expuse public cu un singur set de comenzi, pe orice mașină care are Docker instalat.

Capitolul 5

Modelul de inteligență artificială și setul de date

Componenta de analiză media din WhereWeFishin se bazează pe un model de detecție de obiecte antrenat pe un set de date creat și adnotat manual. Acest capitol detaliază procesul complet: de la colectarea imaginilor și anotarea lor, prin antrenarea și evaluarea modelului, până la integrarea sa în serviciul Python de producție. Această componentă reprezintă contribuția cea mai distinctivă a proiectului față de o aplicație web convențională de rezervări.

5.1 Contextul și motivația unui set de date dedicat

Modelele YOLO preantrenate pe seturi de date publice de referință (precum COCO) nu conțin o clasă dedicată peștilor. Setul COCO, cu cele 80 de categorii ale sale, nu include pești, ceea ce face imposibilă folosirea directă a unui model generic pentru identificarea capturilor. Această absență a justificat construirea unui set de date propriu: fără el, platforma nu ar putea oferi nicio informație despre specia peștelui fotografiat sau filmat.

Alternativele disponibile public, seturi de date de pești pentru medii marine sau subacvatice controlate, au o distribuție a speciilor diferită față de realitatea pescuitului recreativ din România. Specii comune în apele locale, precum crapul, bibanul, șalăul sau știuca, sunt subreprezentate sau absente în aceste seturi. Din acest motiv, crearea unui set de date propriu, adaptat contextului local și condițiilor reale de captare a imaginilor, a fost o alegere necesară, nu opțională.

5.2 Colectarea și structura setului de date

Imaginile au fost colectate din surse variate: fotografii proprii realizate în condiții de pescuit recreativ, capturi de ecran extrase din videoclipuri de pescuit și imagini adnotabile disponibile public. Diversitatea surselor este importantă pentru a evita supraspecializarea modelului pe un singur tip de iluminare, unghi sau fundal.

Setul de date include mai multe clase de specii de pești frecvent întâlnite în contextul pescuitului recreativ, fiecare reprezentată prin imagini cu variații de poziție, unghi de vizualizare, distanță față de cameră și condiții de lumină. Clasele acoperă atât pești

capturați și fotografiați în afara apei, cât și exemplare vizibile la suprafața apei sau în condiții de vizibilitate parțială.

Structura finală a setului de date urmează convenția Ultralytics YOLO: directoarele `train`, `val` și `test` conțin imagini și etichete asociate. Fiecare imagine are un fișier de etichete cu același nume, în format text, unde fiecare linie descrie un obiect prin indexul clasei și coordonatele bounding box-ului normalizate. Această structură este consumată direct de pipeline-ul de antrenare fără preprocesare suplimentară.

5.3 Anotarea și preprocesarea imaginilor

Anotarea bounding box-urilor a fost realizată manual pentru fiecare imagine din setul de date. Instrumentul de adnotare utilizat permite marcarea vizuală a zonelor de interes și exportul direct în formatul YOLO, eliminând conversia manuală a coordonatelor. Anotarea a urmărit principiul delimitării stricte a corpului peștelui, excluzând umbra sau reflecția apei, pentru a reduce zgomotul din semnalul de antrenare.

Preprocesarea și augmentarea datelor au fost aplicate în cadrul pipeline-ului de antrenare. Augmentările includ oglindire orizontală, rotații ușoare, ajustări de luminozitate și contrast, zoom aleatoriu și mozaic, adică combinarea a patru imagini într-una singură, o tehnică specifică familiei YOLO care îmbunătățește detecția obiectelor la scări variate. Aceste transformări măresc artificial diversitatea setului de antrenare și reduc riscul de supraadaptare la condițiile specifice imaginilor originale.

5.4 Antrenarea modelului

Modelul utilizat face parte din familia YOLOv8, implementată prin biblioteca Ultralytics [17]. Alegerea acestei versiuni este justificată de echilibrul dintre acuratețe și viteză de inferență, de suportul nativ pentru antrenare pe GPU și de integrarea facilă cu PyTorch [14]. Familia YOLOv8 menține arhitectura unui singur pas, de tip *single-stage detector*, care estimează simultan localizarea și clasificarea, ceea ce o face potrivită atât pentru imagini, cât și pentru analiza video cadru cu cadru [23].

Antrenarea a pornit de la greutatea preantrenată pe setul COCO, folosind tehnica de transfer learning [20]. Rețeaua a preluat reprezentările generale de viziune din antrenarea inițială și a rafinat straturile finale pe setul de date propriu, adăugând clasele de specii de pești absente din setul original. Această abordare reduce semnificativ numărul de epoci necesare pentru convergență și îmbunătățește stabilitatea antrenamentului față de inițializarea de la zero.

Configurația de antrenare a inclus un număr de epoci ales în funcție de curbele de convergență observate, cu monitorizarea continuă a pierderii de antrenare și a metricilor de validare. S-a utilizat un mecanism de oprire timpurie pentru a preveni supraadaptarea: antrenamentul se oprește automat dacă metrica principală nu se îmbunătățește pe un număr definit de epoci consecutive. Dimensiunea lotului a fost adaptată la memoria GPU disponibilă, iar rata de învățare a urmat un planificator cu coborâre lentă pe parcursul antrenamentului.

5.5 Evaluarea performanței modelului

Evaluarea modelului s-a realizat pe setul de test rezervat, care nu a participat la antrenare sau la selecția hiperparametrilor. Metricile principale utilizate sunt cele standard pentru detecția de obiecte: precizia (P), recall-ul (R) și media preciziei medii la un prag de IoU de 0.5 (mAP@0.5).

Figura 5.1 sintetizează relația dintre principalele metrici de evaluare utilizate în analiza performanței modelului.

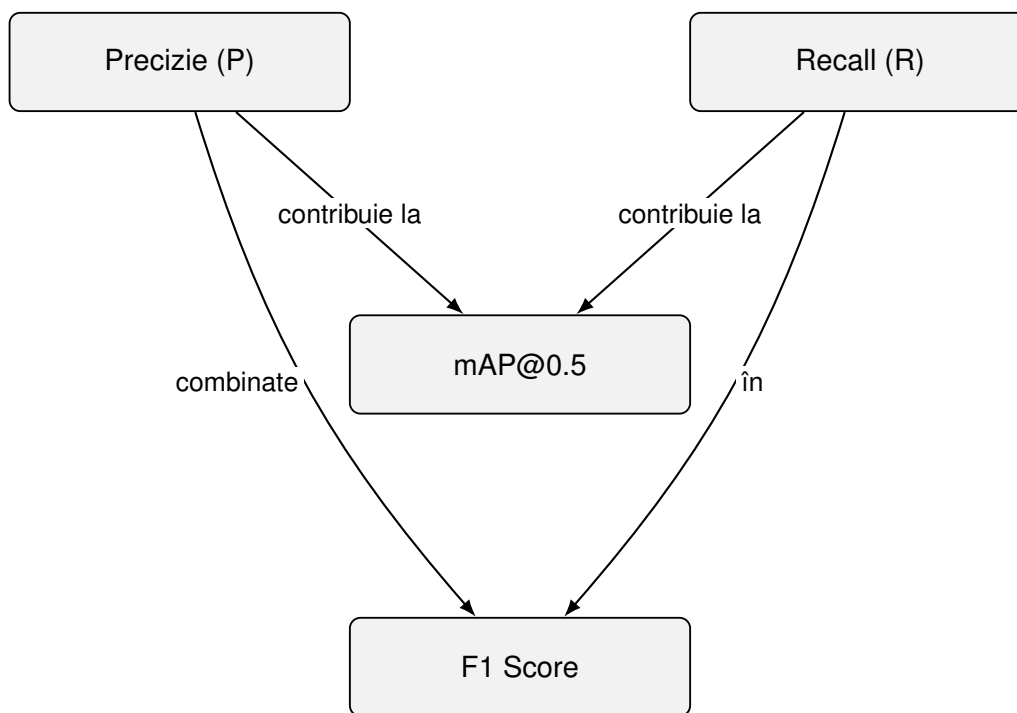


Figura 5.1: Relația dintre metricile principale de evaluare a modelului

Precizia măsoară proporția detecțiilor corecte din totalul detecțiilor efectuate, indicând câte dintre predicțiile modelului sunt reale. Recall-ul măsoară proporția obiectelor reale detectate corect, indicând câți pești au fost identificați din totalul celor prezenți în imagini. mAP@0.5 agregă aceste metrici pe toate clasele și la pragul de IoU de 0.5, oferind o imagine globală a performanței.

Performanța variază între clase, reflectând dificultățile specifice fiecărei specii: dimensiunile tipice ale peștelui în imagine, diversitatea aspectului vizual și frecvența reprezentării în setul de antrenare. Clasele cu mai multe exemple și mai puțină variabilitate morfologică obțin metrici mai ridicate. Clasele mai rare sau cu aspect vizual similar altor specii prezintă valori mai mici, ceea ce indică direcții clare de extindere a setului de date.

Matricea de confuzie a modelului permite identificarea erorilor sistematice: confundarea speciilor cu morfologie similară și detecțiile false în zone cu textură asemănătoare (suprafața apei, vegetație subacvatică). Aceste observații ghidează colectarea ulterioară de date: speciile confundate au nevoie de exemple suplimentare variate, iar imaginile cu fundal complex contribuie direct la reducerea falselor pozitive.

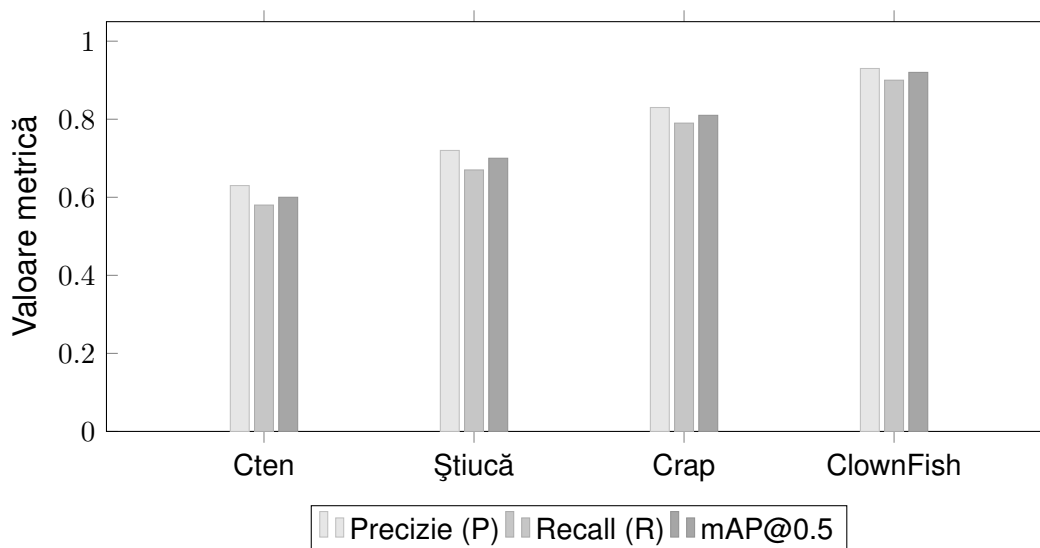


Figura 5.2: Precizie, Recall și mAP@0.5 per clasă pe setul de test

5.6 Integrarea modelului antrenat în producție

Modelul antrenat este exportat în formatul .pt (PyTorch checkpoint) și inclus în directorul serviciului Python Flask. La pornirea serviciului, modelul este încărcat o singură dată în memorie și reutilizat pentru toate cererile de analiză, evitând overhead-ul repetitiv al inițializării.

Pragul de încredere pentru detecție este configurat explicit și a fost ales pe baza curbelor de precizie-recall observate pe setul de validare: o valoare prea mică include detecții nesigure care produc zgomot vizual, iar o valoare prea mare exclude detecții corecte cu grad de încredere moderat. Pragul ales reprezintă un compromis care favorizează precizia față de recall, deoarece în contextul aplicației o detecție incertă este mai problematică decât o detecție lipsă.

Pentru analiza video, modelul este aplicat cadru cu cadru, iar rezultatele sunt transmise tracker-ului ByteTrack [26] care menține identitatea obiectelor de-a lungul secvenței. Această separare clară între detecție și tracking permite înlocuirea sau actualizarea modelului de detecție fără a modifica logica de urmărire temporală și invers.

Serviciul expune un endpoint de inferență care primește un fișier media, determină tipul de analiză (image sau video) pe baza tipului MIME și returnează un răspuns structurat cu detecțiile, statisticile și, în cazul video, un fișier reîncodat cu adnotările suprapuse.

```

1 model = YOLO("model.pt") # incarcata o singura data la pornire
2
3 @app.route("/analyze", methods=["POST"])
4 def analyze():
5     if "file" not in request.files:
6         return jsonify({"error": "No file provided"}), 400
7
8     file = request.files["file"]
9     mime = file.content_type
10
11     if mime.startswith("image/"):

```

```

12     results = run_image_inference(file, model)
13     elif mime.startswith("video/"):
14         results = run_video_inference(file, model)
15     else:
16         return jsonify({"error": "Unsupported media type"}), 415
17
18     return jsonify({
19         "detections": results["detections"],
20         "species_summary": results["species_summary"],
21         "total_count": results["total_count"],
22     })

```

Listing 5.1: Endpoint Flask pentru inferența modelului YOLO

5.7 Limitări și observații ale modelului

Performanța modelului este influențată de condițiile în care sunt realizate fotografiile sau videoclipurile. Imaginile cu iluminare slabă, reflexii puternice pe suprafața apei sau cu peștii parțial ieșiți din cadru produc detecții cu grad de încredere mai scăzut sau false negative. Aceste limite sunt specifice sistemelor de viziune artificială și nu pot fi eliminate complet fără extinderea setului de antrenare cu exemple specifice acestor condiții.

O altă limită practică este că modelul a fost antrenat pe specii frecvente în contextul pescuitului recreativ local. Specii mai rare sau exotice nu sunt recunoscute și vor genera fie nicio detecție, fie o clasificare incorectă spre cea mai apropiată clasă disponibilă. Extinderea setului de date cu specii suplimentare reprezintă o direcție naturală de îmbunătățire.

Dimensiunea setului de date, deși suficientă pentru un prototip funcțional, rămâne mai mică față de seturile folosite pentru antrenarea modelelor generice. Această limitare este compensată parțial de transfer learning, dar are impact vizibil pe clasele cu mai puține exemple. Colectarea continuă de imagini în condiții variate este cea mai eficientă modalitate de a îmbunătăți robustețea modelului pe termen lung.

5.8 Sinteza capitolului

Capitolul a descris procesul complet de construire și integrare a componentei de inteligență artificială din WhereWeFishin: de la motivația unui set de date propriu și anotarea manuală a imaginilor, prin antrenarea modelului YOLOv8 cu transfer learning, evaluarea performanței pe setul de test și integrarea modelului în serviciul Python de producție. Contribuția personală se regăsește în fiecare etapă: selectarea și adnotarea imaginilor, configurarea antrenamentului, calibrarea pragurilor de inferență și integrarea modelului în arhitectura serviciului. Aceasta este zona proiectului care transformă platforma dintr-o aplicație de rezervări într-un instrument cu valoare analitică adăugată pentru utilizatorii săi.

Capitolul 6

Interfața utilizatorului și experiența de utilizare

Acest capitol descrie modul în care funcționalitățile implementate sunt prezentate utilizatorului prin interfața web. Dacă în capitolul anterior am detaliat deciziile tehnice din spatele fiecărei componente, acum mă concentrez pe experiența concretă: cum navighează un utilizator prin aplicație, ce funcții sunt disponibile pentru fiecare rol și cum contribuie deciziile de interfață la o utilizare naturală și eficientă a platformei.

Interfața WhereWeFishin este o aplicație Angular de tip SPA, cu rutare lazy și componente standalone. Structura sa urmărește separarea clară a rolurilor: fiecare categorie de utilizator are zone proprii, accesibile prin rute protejate și cu un set delimitat de funcții disponibile. Separarea nu este doar de securitate, ci și de claritate: utilizatorul obișnuit nu vede funcțiile administrative, iar managerul nu este expus la zona de supraveghere a administratorului.

6.1 Principii de organizare a interfeței

Proiectarea interfeței a pornit de la câteva constrângeri clare. Aplicația trebuie să fie utilizabilă de persoane fără experiență tehnică, mai precis pescari care caută un loc de pescuit și vor să facă o rezervare cât mai simplu posibil. Totodată, trebuie să ofere un panou de control util pentru manageri, care au nevoie de acces rapid la informații operaționale. Ambele cerințe trebuie satisfăcute de aceeași aplicație, ceea ce impune o separare clară a zonelor și o navigare intuitivă.

Structura generală urmează o ierarhie de trei niveluri. La primul nivel se află zonele mari ale aplicației: zona publică (hartă și detalii locații), zona utilizatorului autentificat (rezervări, analiză media, profil) și zonele administrative (dashboard manager, panou angajat, control administrator). La al doilea nivel sunt secțiunile din cadrul fiecărei zone. La al treilea nivel se află componentele interactive: formulare, calendare, hărți și liste de rezultate.

Navigarea principală este realizată printr-o bară de meniu care reflectă starea de autentificare și rolul utilizatorului curent. Un vizitator neautentificat vede doar opțiunile de explorare și autentificare. După autentificare, meniul se adaptează automat în funcție de rolul acordat. Această abordare reduce confuzia și limitează accesul vizual la funcții irelevante pentru un anumit tip de utilizator. Tranzițiile de navigare sunt fluide datorită rutării lazy: fiecare zonă funcțională se încarcă doar la prima accesare, reducând timpul

inițial de pornire al aplicației.

Feedback-ul vizual pentru operații asincrone (upload, procesare media, confirmare rezervare) este furnizat prin indicatori de stare clari, astfel încât utilizatorul să nu rămână fără informații în timp ce o operație se desfășoară în fundal. Mesajele de eroare sunt formulate pentru a fi utile și specifice, nu generice, ghidând utilizatorul spre acțiunea corectă în loc să blocheze fluxul.

6.2 Experiența utilizatorului obișnuit

Fluxul principal al unui utilizator obișnuit începe cu harta interactivă. Aceasta este pagina centrală de explorare și reprezintă punctul de intrare în platformă. Pe hartă sunt marcate toate locațiile active, fiecare cu un marker personalizat. La interacțiunea cu un marker apare o fereastră informativă cu datele esențiale: denumire, rating mediu, tarif orar și un buton de acces la detalii complete. Geolocalizarea utilizatorului permite orientarea rapidă față de locațiile din apropiere.

Pagina de detalii a unei locații reunește toate informațiile relevante: descrierea locației, lista pontoanelor disponibile, recenziile altor utilizatori și un modul de rezervare. Utilizatorul poate selecta un ponton, alege data și intervalul orar dorit și vede costul calculat automat pe baza tarifului orar și a duratei. Calendarul de rezervare evidențiază zilele parțial sau complet ocupate și blochează selecțiile invalide, ghidând utilizatorul spre intervale disponibile fără a necesita verificări manuale.

Fluxul de rezervare se finalizează prin plată online sau prin confirmare directă, în funcție de configurația locației. Integrarea cu Stripe este transparentă pentru utilizator: datele de plată sunt completate într-un formular securizat integrat în pagină, fără redirectionare externă. După confirmare, sesiunea apare în istoricul utilizatorului și generează un cod QR asociat rezervării, accesibil direct din aplicație și pregătit pentru verificarea la fața locului.

Modulul de analiză media este accesibil dintr-o secțiune dedicată. Utilizatorul poate încărca o imagine sau un videoclip, iar interfața afișează starea de procesare în timp real printr-un mecanism de polling vizibil. Rezultatele includ speciile detectate, numărul de instanțe identificate și, în cazul videoclipurilor, un rezumat al detecțiilor pe intervale de timp. Componenta este separată intenționat de fluxul de rezervare, pentru a nu complica experiența utilizatorilor care nu doresc să folosească funcția de analiză media.

Secțiunea de profil și istoricul activității oferă utilizatorului o imagine completă a rezervărilor trecute și viitoare, a analizelor media efectuate și a recenziilor lăsate. Această zonă îndeplinește și un rol de trasabilitate: utilizatorul poate oricând verifica detaliile unei rezervări, inclusiv codul QR, fără a depinde de un email de confirmare.

6.3 Interfața managerului și dashboard-ul operațional

Zona managerului este construită în jurul unui dashboard care concentrează informațiile operaționale relevante pentru administrarea unui loc de pescuit. Structura este organizată pe secțiuni distincte: administrarea pontoanelor, gestionarea angajaților, informații despre locație și rezervările active.

Administrarea pontoanelor utilizează direct harta interactivă: managerul poate adăuga un ponton nou printr-un instrument de desen pe hartă, conturând vizual zona rezervabilă. Coordonatele sunt extrase automat din interacțiunea cu harta, fără a

fi nevoie de introducerea manuală a valorilor numerice. Această abordare reduce ambiguitățile și oferă un feedback imediat despre cum va apărea zona în interfața publică, permițând ajustări înainte de salvare.

Gestionarea angajaților permite asocierea utilizatorilor la locație, cu drepturi limitate la operațiile de verificare a sesiunilor. Adăugarea și eliminarea angajaților se face direct din dashboard, fără procese administrative suplimentare. Rezervările active sunt prezentate într-o listă filtrabilă, cu informații despre utilizator, ponton, interval și starea curentă. Managerul poate urmări sesiunile planificate, poate vizualiza confirmările de prezență și poate monitoriza activitatea locației în timp real.

Procesul de aplicare pentru rolul de manager este mediat de un wizard cu mai mulți pași: date descriptive ale locației, poziționare pe hartă și o pagină de revizuire finală înainte de trimitere. Formatul ghidat reduce erorile de completare și permite validarea datelor pe parcurs. Solicitantul poate urmări starea aplicației sale și, în cazul respingerii, poate vizualiza motivul primit de la administrator și poate retrimite cererea cu informațiile corectate.

6.4 Experiența angajatului și verificarea prin cod QR

Angajatul are cel mai restrâns set de funcții, adaptat nevoilor operaționale de la fața locului. Interfața sa principală este componenta de scanare a codului QR, accesibilă direct după autentificare, fără pași intermediari.

Componenta de scanare utilizează camera dispozitivului și procesează codul QR al sesiunii de pescuit. Odată identificată sesiunea, interfața afișează detaliile rezervării: utilizatorul, pontonul, intervalul și starea curentă. Dacă sesiunea este validă și în intervalul corect, angajatul confirmă prezența cu o singură acțiune. Dacă există o neconcordanță, de exemplu sesiune expirată, ponton diferit sau cod invalid, interfața afișează un mesaj explicit care descrie problema, fără a bloca complet fluxul.

Această abordare este importantă pentru utilizarea în teren. Angajatul lucrează adesea în condiții care nu permit o interacțiune lungă cu aplicația, de exemplu în aer liber, cu posibilă iluminare variabilă sau conexiune mobilă instabilă. Simplificarea interfeței la funcțiile strict necesare pentru verificare este o decizie deliberată de UX, care prioritizează viteza și claritatea operației în fața complexității vizuale. Angajatul poate verifica o sesiune în câteva secunde, fără a naviga prin meniuri sau a căuta manual rezervarea în liste.

6.5 Panoul administrativ

Administratorul platformei are acces la o zonă separată, cu funcții de supraveghere globală. Principalele secțiuni vizează gestionarea utilizatorilor înregistrați, monitorizarea locurilor de pescuit active și procesarea aplicațiilor pentru rolul de manager.

Fluxul de aprobare a aplicațiilor este direct și bine delimitat: administratorul vede lista cererilor primite, poate accesa detaliile fiecăreia, inclusiv datele descriptive ale locației propuse și informațiile despre solicitant, și ia o decizie de aprobare sau respingere. În cazul respingerii, câmpul de motivație permite transmiterea unui feedback util solicitantului, reducând iterațiile de comunicare. O cerere aprobată transformă automat utilizatorul în manager, creează locul de pescuit în platformă și îl face vizibil pe hartă.

Zona administrativă oferă și o vizualizare de ansamblu a utilizatorilor și locațiilor active. Administratorul poate interveni în cazuri excepționale (suspendarea unui cont sau dezactivarea unei locații) fără a depinde de acțiuni externe. Această funcție de supraveghere nu înlocuiește un instrument dedicat de monitorizare, dar oferă suficiente informații pentru gestionarea curentă a platformei în stadiul actual de dezvoltare.

6.6 Gestionarea autentificării și comunicarea cu API-ul

O aplicație Angular de tip SPA care consumă un API securizat cu JWT trebuie să rezolve mai multe probleme tehnice: stocarea tokenului, atașarea lui la fiecare cerere, protejarea rutelor private și gestionarea expirării sesiunii.

Tokenul JWT primit la autentificare este stocat în `localStorage`, ceea ce îl face persistent între sesiuni de browser. La fiecare cerere HTTP, un *interceptor* Angular atașează automat header-ul `Authorization: Bearer <token>` la toate cererile destinate API-ului. Interceptorul verifică prezența tokenului și îl adaugă fără nicio intervenție explicită în componentele care inițiază cererile. Dacă API-ul returnează HTTP 401, interceptorul poate declanșa delogarea automată și redirecționarea spre pagina de autentificare.

Protecția rutelor private este realizată prin *route guards*. Fiecare zonă funcțională, anume dashboard manager, panou angajat și control administrator, are un guard care verifică dacă utilizatorul este autentificat și dacă rolul său corespunde zonei accesate. Dacă nu, Angular redirecționează automat spre o pagină publică, fără a încărca componenta protejată. Această verificare se face pe baza datelor din token, decodate local la nivel de client: validarea criptografică rămâne exclusiv în responsabilitatea backend-ului.

Serviciile Angular pentru comunicarea cu API-ul sunt organizate pe domenii: un serviciu pentru autentificare, unul pentru rezervări, unul pentru locuri de pescuit și unul pentru analiza media. Fiecare serviciu expune metode tipizate care returnează `Observable<T>`, permițând componentelor să se aboneze la răspunsuri și să gestioneze stările de încărcare, succes și eroare în mod declarativ. Această arhitectură facilitează și testarea unitară a logicii de business din frontend, izolat de comunicarea HTTP.

6.7 Adaptabilitate și considerații de utilizare mobilă

Interfața a fost construită cu atenție la comportamentul pe ecrane de dimensiuni diferite. Componentele principale (harta, formularele, listele de rezervări și modulele de analiză media) sunt responsabile și se adaptează la rezoluțiile uzuale ale dispozitivelor mobile și desktop. Acest aspect este important mai ales pentru angajați și utilizatori care accesează aplicația de pe telefon în timp ce se află la locație.

Componenta de scanare QR, utilizată în principal pe dispozitive mobile, a beneficiat de o atenție specială în ceea ce privește toleranța la erori. Implementarea tratează variații ale structurii datelor din cod și gestionează situațiile în care camera nu identifică imediat codul, fără a întrerupe fluxul angajatului. Paginarea listelor de rezultate media și încărcarea întârziată a elementelor video contribuie la o experiență mai fluidă pe conexiuni cu lățime de bandă limitată.

Navigarea pe dispozitive mici a impus și adaptarea meniului și a zonelor de acțiune: elementele interactive au dimensiuni suficiente pentru interacțiune tactilă, iar structura paginilor evită derularea excesivă. Interfața nu implementează funcționalitate offline

completă, dar arhitectura sa modulară, bazată pe componente standalone Angular, oferă o bază bună pentru extinderea ulterioară în direcția unui comportament de tip PWA.

6.8 Sinteza capitolului

Interfața WhereWeFishin reflectă structura multi-rol a platformei și prioritizează claritatea față de complexitatea vizuală. Fiecare tip de utilizator are o zonă proprie, cu funcțiile necesare activității sale, fără a fi expus la elemente irelevante. Deciziile de UX au urmărit susținerea fluxurilor reale de utilizare: de la explorarea unui loc de pescuit pe hartă și finalizarea unei rezervări, până la analiza media asistată de inteligență artificială și verificarea prezenței pe teren prin cod QR. Feedback-ul clar pentru operațiile asincrone și adaptabilitatea la dispozitive mobile completează experiența, asigurând o platformă utilizabilă în scenarii variate de acces și context.

Capitolul 7

Testare și evaluare

După prezentarea implementării, pasul următor este verificarea comportamentului aplicației prin teste și validări operaționale. Într-un sistem care combină autentificare, rezervări, administrare multi-rol, integrare cu servicii externe și procesare media, testarea nu poate fi redusă la verificarea unor metode izolate. Este necesară o abordare care acoperă atât validarea locală a regulilor de business, cât și comportamentul compus al aplicației în scenarii apropiate de utilizarea reală.

Capitolul este organizat în jurul infrastructurii de testare existente în proiect și al observațiilor rezultate din rularea și analiza componentelor implementate. Accentul principal este pe backend, unde există cea mai consistentă suită de teste automate, dar sunt discutate și acoperirea redusă din frontend, validările infrastructurale ale serviciului Python și limitele actuale ale procesului de testare.

7.1 Strategia de testare și infrastructura utilizată

Strategia de testare este centrată pe backend-ul .NET, unde am construit o infrastructură clară pentru teste unitare, teste de controller și teste de integrare. Proiectul dedicat testelor folosește framework-ul xUnit [18] și include dependențe pentru mocking, pentru rularea aplicației în regim de test și pentru colectarea de coverage. Această configurație permite validarea atât a componentelor individuale, cât și a fluxurilor complete la nivel HTTP, fără a depinde de o instanță externă sau de o bază de date de producție.

La momentul evaluării, suita backend a raportat **349 de teste trecute și 0 teste eșuate**. Există 22 de fișiere de test distribuite pe mai multe categorii: validarea DTO-urilor, testarea serviciilor, testarea controllerelor, testarea repository-urilor și teste de integrare.

Un rol central îl are fabrica de aplicație folosită pentru testele de integrare. Aceasta pornește API-ul în mediul *IntegrationTesting*, injectează configurări specifice pentru conexiunea la bază de date, pentru JWT și pentru integrarea cu serviciul de recunoaștere, iar înlocuirea serviciului de email cu o implementare neutră permite verificarea fluxurilor principale fără efecte externe nedorite. Baza de date *InMemory* asigură izolarea între teste și reduce costul de execuție.

Infrastructura de testare este importantă deoarece permite verificarea aplicației la mai multe niveluri fără a dubla artificial codul de test.

7.2 Rezultatele rulării suitei de teste

Suita de teste backend a fost rulată integral în momentul redactării acestui capitol. Rezultatul a fost stabil: 349 de teste trecute și niciun test eșuat. Chiar dacă un astfel de rezultat nu garantează absența completă a problemelor, arată că funcționalitățile importante din zona de backend sunt acoperite suficient de bine pentru o dezvoltare controlată.

Suita nu este concentrată într-o singură zonă, ci distribuită pe mai multe niveluri: validarea DTO-urilor, testarea serviciilor, a controllerelor, a repository-urilor și a fluxurilor de integrare. Tabelul 7.1 prezintă distribuția actuală a fișierelor de test.

Categorie	Număr fișiere de test
DTO-uri și validare	1
Servicii	2
Controllere	11
Repository-uri	2
Teste de integrare	5
Extensii și infrastructură	1
Total	22

Tabela 7.1: Structura actuală a suitei de teste din proiectul backend

Figura 7.1 prezintă distribuția estimată a testelor pe categorii, pe baza numărului de fișiere și a densității medii de teste per fișier.

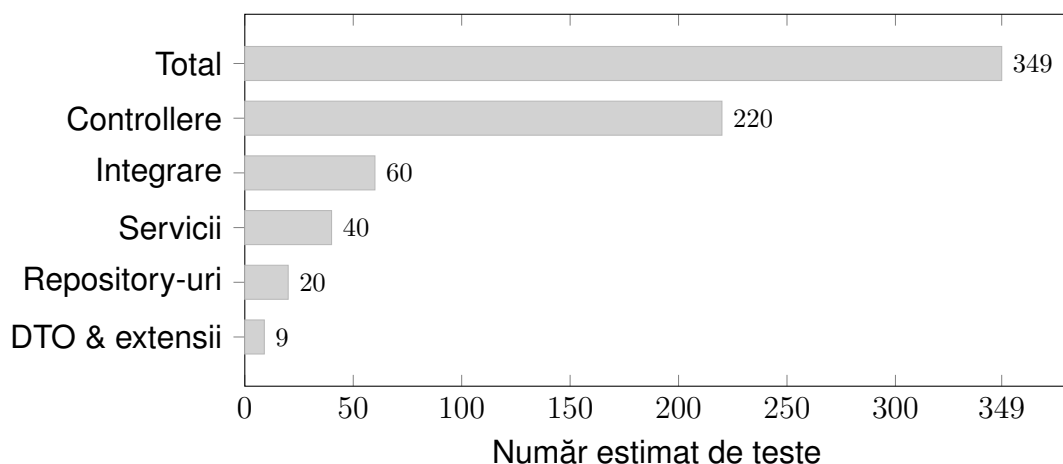


Figura 7.1: Distribuția estimată a testelor pe categorii în proiectul backend

Un exemplu clar sunt testele de integrare pentru autentificare: verifică înregistrarea unui utilizator, tratarea conflictelor de identitate, autentificarea și validarea tokenului primit. Similar, testele pentru aplicațiile de manager urmăresc scenarii complete: crearea cererii, respingerea, retransmiterea după corectare și aprobarea finală care promovează utilizatorul. Aceste scenarii sunt valoroase deoarece verifică împreună reguli de business, persistență și control al accesului.

O altă zonă bine reprezentată este analiza media. Testele pentru serviciul de recunoaștere validează atât cazul în care serviciul Python răspunde corect, cât și scenariile în care răspunsul extern este invalid sau serviciul nu poate fi contactat.

7.3 Testarea unitară și validarea datelor

Primul nivel de validare este oferit de testele unitare orientate spre DTO-uri și reguli de validare. Conform literaturii de specialitate, aceste teste sunt utile pentru verificarea locală a comportamentului unor componente mici, înainte ca sistemul să fie evaluat în fluxuri mai largi [21].

Testele verifică că obiectele folosite în autentificare, rezervări, recenzii, locuri de pescuit sau pontoane respectă constrângerile declarate. Se utilizează scenarii parametrizate pentru a acoperi combinații variate de date valide și invalide: lungimea credențialelor, coerența dintre parolă și confirmare, limitele pentru durata rezervării, intervalul permis pentru rating sau corectitudinea coordonatelor.

Pe lângă validarea DTO-urilor, testele unitare vizează și servicii cu logică mai densă: autentificarea și integrarea cu serviciul de analiză video. Acestea verifică comportamentul local al componentelor, izolat de infrastructura HTTP sau de persistență.

7.4 Testarea controllerelor și a comportamentului API

Al doilea nivel de validare este oferit de testele dedicate controllerelor. Acestea verifică direct răspunsurile HTTP, codurile de stare și reacția API-ului la cereri valide sau invalide, fără a porni întregul sistem în forma completă a testelor de integrare. Rolul lor este să confirme că punctele de intrare ale backend-ului respectă contractele așteptate și că tratează corect atât situațiile reușite, cât și erorile.

Pentru WhereWeFishin, această zonă este relevantă mai ales pentru controllerele cu logică operațională densă: autentificare, rezervări, administrarea locațiilor și analiza video. În cazul rezervărilor, testarea controllerului permite verificarea interpretării datelor de intrare, a reacției la conflicte de disponibilitate și a integrării cu componentele auxiliare. Controllerele de analiză video pot fi validate din perspectiva accesului autorizat, a limitelor de upload și a structurii răspunsurilor.

Existența acestui nivel intermediar reduce riscul ca problemele de contract API să fie descoperite abia în testele de integrare sau în utilizarea manuală.

7.5 Testarea de integrare a fluxurilor critice

Zona cu cea mai mare valoare pentru evaluarea aplicației este cea a testelor de integrare. Acestea exercită API-ul prin cereri HTTP reale și validează atât răspunsurile backend-ului, cât și efectele produse asupra datelor persistente.

Fluxul complet de autentificare este unul dintre primele testate: înregistrare, validarea persistenței, autentificare și verificarea tokenului. Sunt acoperite și cazuri negative: duplicarea username-ului sau a adresei de email, date invalide.

Workflow-ul de aplicare pentru manager este un al doilea exemplu important. Testele validează nu doar structura răspunsurilor API, ci și corectitudinea tranzițiilor de stare și impactul deciziilor administrative asupra datelor persistente. Acesta este unul dintre cele mai bune exemple că logica de business din backend este verificată pe scenarii complete, nu doar pe fragmente de cod.

Fluxurile legate de rezervări și roluri operaționale completează tabloul. Chiar dacă nu toate scenariile au aceeași adâncime de acoperire, prezența testelor pentru booking

și pentru module precum angajații sau administrarea platformei arată că sistemul este evaluat și în situațiile în care mai multe componente trebuie să colaboreze.

7.6 Validarea operațională a comportamentului sistemului

Dincolo de testele automate, evaluarea aplicației include și mecanismele de protecție configurate direct în infrastructura runtime. Acestea oferă o formă de validare operațională complementară testării clasice.

Un prim element este politica strictă asociată JWT-urilor: validarea explicită a issuer-ului, audience-ului, semnăturii și duratei de viață, cu o fereastră de toleranță nulă pentru expirare. Endpoint-urile sensibile de autentificare sunt protejate și prin rate limiting.

Un al doilea element este legat de gestionarea cererilor voluminoase și a fluxurilor media. Limitarea la 150 MB, împreună cu configurarea timeout-urilor și a conexiunilor, oferă un cadru mai stabil pentru încărcarea videoclipurilor. Compresia răspunsurilor, caching-ul și mecanismele de retry pentru conexiunile la baza de date contribuie la o aplicație mai stabilă în condiții reale.

Această evaluare operațională nu înlocuiește testele automate, dar arată că soluția a fost construită și cu gândul la securitate, trafic și reziliență.

7.7 Acoperire actuală, limitări și direcții de extindere

O evaluare realistă trebuie să evidențieze și limitele actuale. Backend-ul are cea mai bună acoperire, datorită combinației dintre teste unitare, teste de controller și teste de integrare. Frontendului îi corespunde un singur fișier de test automat, dedicat serviciului pentru locurile de pescuit. Serviciul Python dispune mai ales de verificări infrastructurale și de tip smoke test, nu de o suită funcțională completă.

Această distribuție inegală este explicabilă: complexitatea logicii de business este concentrată în backend. Totuși, rămâne o limitare reală. Multe dintre fluxurile frontend sunt validate prin integrarea cu backend-ul și prin verificări manuale. Analiza media depinde într-o măsură mai mare de observarea comportamentului în execuție.

Altă limitare este absența unor măsurători explicite de performanță sau a unor teste end-to-end complete care să includă simultan interfața, backend-ul și serviciile externe.

Ca direcții de extindere: o suită mai consistentă de teste frontend, teste automate pentru pipeline-ul Python și generarea sistematică a unor rapoarte de coverage. Proiectul include deja dependențele necesare pentru colectarea coverage-ului, dar un raport numeric complet nu a fost generat în această evaluare.

7.8 Sinteza capitolului

Testarea și evaluarea WhereWeFishin arată că aplicația beneficiază de o bază solidă de validare la nivel de backend, cu acoperire bună pentru validarea datelor, comportamentul controllerelor și fluxurile de integrare importante. Infrastructura construită în jurul xUnit și a fabricii de aplicație permite verificarea unor scenarii realiste.

În același timp, evaluarea evidențiază limitele: acoperire automată redusă în frontend, validare limitată a serviciului Python și absența unor măsurători de performanță mai

ample. Aceste observații nu diminuează valoarea implementării, ci oferă o imagine realistă asupra nivelului tehnic atins și a pașilor care pot urma.

Capitolul 8

Securitate și infrastructură de deployment

Funcționalitățile vizibile ale unei aplicații web, mai precis formulare, hărți și rezervări, se sprijină pe o infrastructură de securitate care, dacă este neglijată, poate compromite întregul sistem. Acest capitol tratează în detaliu mecanismele de securitate implementate în WhereWeFishin și modul în care aplicația este containerizată și configurată pentru deployment. Sunt analizate autentificarea prin JWT, autorizarea pe roluri, protecția fluxurilor sensibile, izolarea componentelor și gestionarea configurației pe medii.

8.1 Arhitectura de securitate în straturi

Securitatea aplicației nu este concentrată într-un singur punct, ci distribuită pe mai multe niveluri independente. Această abordare în straturi reduce riscul că un singur mecanism compromis să afecteze întregul sistem.

La nivel de transport, comunicarea între client și backend se realizează exclusiv prin HTTPS, ceea ce elimină riscul interceptării tokenurilor sau a datelor sensibile în tranzit. La nivel de autentificare, identitatea utilizatorului este verificată la fiecare cerere prin validarea unui token JWT. La nivel de autorizare, fiecare endpoint controlează explicit ce rol are voie să execute operația respectivă. La nivel de logică de business, serviciile verifică și apartenența resurselor: un manager poate modifica doar locațiile proprii, nu pe ale altora.

Un principiu de bază aplicat este că frontendului nu i se acordă încredere pentru decizii de securitate. Orice acțiune sensibilă este revalidată complet în backend, indiferent de ce afirmă sau trimite clientul. Această separare este importantă deoarece interfața web poate fi ocolită de oricine poate construi o cerere HTTP manuală [2].

8.2 Autentificarea prin JSON Web Tokens

Autentificarea în WhereWeFishin folosește tokenuri JWT, conform RFC 7519 [1]. Un token JWT este un șir compact format din trei segmente codificate Base64URL, separate prin puncte: antet, payload și semnătură.

Antetul specifică algoritmul de semnare și tipul tokenului. În implementarea curentă se folosește HMAC-SHA256 (HS256), un algoritm simetric care semnează și verifică

tokenul cu aceeași cheie secretă stocată pe server. Payload-ul conține *claims*, adică perechi cheie-valoare cu informații despre utilizator și token:

- sub: identificatorul unic al utilizatorului;
- email: adresa de e-mail, folosită și ca username implicit;
- role: rolul acordat (User, Employee, Manager, Administrator);
- iss și aud: emițătorul și destinatarul așteptat al tokenului;
- exp și iat: momentul expirării și al emiterii.

Semnătura este calculată ca HMAC-SHA256 aplicat concatenării antet și payload. Orice modificare a conținutului tokenului invalidează semnătura, ceea ce înseamnă că un atacator nu poate modifica rolul sau ID-ul utilizatorului fără a cunoaște cheia secretă.

În ASP.NET Core [7], validarea JWT este configurată explicit prin parametrii de validare injectați la înregistrarea middleware-ului: issuer-ul, audience-ul, durata de viață și cheia de semnare sunt verificate la fiecare cerere. Fereastra de toleranță pentru expirare (*ClockSkew*) este setată la zero, ceea ce înseamnă că un token expirat este respins imediat, fără perioadă de grație. Odată validat, tokenul este decodat, iar claims-urile sunt populate în `HttpContext.User`, disponibile în orice punct al pipeline-ului.

```
1 builder.Services
2   .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
3   .AddJwtBearer(options =>
4     {
5       options.TokenValidationParameters = new
6         TokenValidationParameters
7         {
8           ValidateIssuer          = true,
9           ValidateAudience       = true,
10          ValidateLifetime        = true,
11          ValidateIssuerSigningKey = true,
12          ValidIssuer             = builder.Configuration["Jwt:Issuer"],
13          ValidAudience          = builder.Configuration["Jwt:Audience"],
14          IssuerSigningKey        = new SymmetricSecurityKey(
15            Encoding.UTF8.GetBytes(
16              builder.Configuration["Jwt:Key"]!)),
17          ClockSkew = TimeSpan.Zero
18        }
19    });
```

Listing 8.1: Configurarea autentificării JWT în Program.cs

8.3 Autorizarea pe bază de roluri și verificarea proprietății resurselor

Autentificarea confirmă identitatea utilizatorului. Autorizarea decide ce operații are voie să execute. WhereWeFishin combină două mecanisme: autorizarea declarativă pe roluri și verificarea explicită a proprietății resurselor.

Autorizarea declarativă se aplică la nivel de controller sau de acțiune prin atributul `[Authorize]`, cu specificarea rolului sau rolurilor permise. Un endpoint de administrare a locației este decorat cu `[Authorize(Roles = "Manager,Administrator")]`, ceea ce

face ca framework-ul să respingă automat orice cerere de la un utilizator fără unul dintre aceste roluri, înainte ca logica de business să fie executată.

```
1 [ApiController]
2 [Route("api/[controller]")]
3 [Authorize(Roles = "Manager,Administrator")]
4 public class FishingSpotsController : ControllerBase
5 {
6     [HttpPut("{id}")]
7     public async Task<IActionResult> Update(
8         Guid id, UpdateFishingSpotDto dto)
9     {
10         // Extragere identitate din claims JWT
11         var userId = User.FindFirstValue(
12             ClaimTypes.NameIdentifier);
13
14         var spot = await _service.GetByIdAsync(id);
15
16         // Verificare proprietate resursa
17         if (spot.ManagerId.ToString() != userId)
18             return Forbid();
19
20         await _service.UpdateAsync(id, dto);
21         return NoContent();
22     }
23 }
```

Listing 8.2: Autorizare pe rol și verificarea proprietății resursei

Autorizarea declarativă nu este suficientă pentru resurse care aparțin unui anumit utilizator. Un manager autentificat nu trebuie să poată modifica locația unui alt manager, chiar dacă ambii au rolul `Manager`. Acest control este aplicat în stratul de servicii: ID-ul utilizatorului curent este extras din claims-urile tokenului și comparat cu câmpul de proprietate al resursei solicitate. Dacă nu coincid, cererea este respinsă cu HTTP 403, indiferent de rolul utilizatorului.

Fluxul complet al unei cereri de actualizare a unei locații este astfel: validare JWT → extragere claims → verificare rol `Manager` → verificare că locația aparține managerului autentificat → execuție logică de business. Fiecare pas este independent, iar eșecul oricăruia oprește procesarea fără a expune informații suplimentare.

8.4 Protecția fluxurilor sensibile

Endpoint-urile de autentificare sunt protejate și prin *rate limiting*. ASP.NET Core oferă, începând cu versiunea 7, middleware dedicat pentru limitarea numărului de cereri pe interval de timp. Această măsură previne atacurile de tip brute-force: un atacator care încearcă parole în mod repetat va primi răspunsuri de tip HTTP 429 după depășirea pragului, fără posibilitatea de a continua cu cereri nelimitate.

Validarea datelor de intrare este aplicată sistematic prin *data annotations* pe clasele DTO. Atributele `[Required]`, `[StringLength]`, `[Range]` și `[RegularExpression]` definesc constrângerile așteptate direct pe modelul de date. ASP.NET Core evaluează aceste constrângeri automat în procesul de model binding: o cerere cu date invalide primește HTTP 400 cu detalii despre câmpurile incorecte, fără ca logica de business să fie invocată.

```

1 public class CreateFishingSessionDto
2 {
3     [Required]
4     public Guid FishingSpotId { get; set; }
5
6     public Guid? PontoonId { get; set; }
7
8     [Required]
9     public DateTime StartTime { get; set; }
10
11    [Required]
12    public DateTime EndTime { get; set; }
13 }
14
15 public class CreateReviewDto
16 {
17     [Required]
18     public Guid FishingSpotId { get; set; }
19
20     [Required]
21     [Range(1, 5, ErrorMessage = "Rating must be between 1 and 5")]
22     public int Rating { get; set; }
23
24     [StringLength(1000)]
25     public string? Comment { get; set; }
26 }

```

Listing 8.3: Exemplu de DTO cu validare declarativă prin DataAnnotations

Validarea fișierelor media este tratată în două straturi. La nivel de server (Kestrel), limita de 150 MB pentru corpul cererii este configurată explicit, respingând fișierele prea mari înainte ca acestea să fie procesate. La nivel de controller, tipul MIME declarat este verificat față de o listă albă de formate acceptate. Fișierul nu este scris în stocare permanentă până când toate validările nu au trecut.

8.5 Izolarea componentelor și securitatea comunicației interne

Microserviciul Python de analiză media nu este expus direct în rețeaua publică. El rulează pe un port intern, accesibil exclusiv de la backend-ul .NET. Prin această izolare, toate verificările de autentificare și autorizare sunt aplicate de API-ul principal înainte ca orice date să ajungă la serviciul de procesare. Serviciul Python nu implementează autentificare proprie: această responsabilitate rămâne complet la nivelul backend-ului.

Secretele aplicației, printre care cheia de semnare JWT, string-ul de conexiune la baza de date, cheia API Stripe și credențialele SMTP, nu sunt stocate în codul sursă. Acestea sunt furnizate prin variabile de mediu sau prin fișiere de configurare excluse din sistemul de versionare. Separarea configurației de cod permite utilizarea unor valori diferite pe medii diferite fără modificarea aplicației.

8.6 Containerizarea aplicației cu Docker

Deploymentul aplicației WhereWeFishin folosește Docker [8] și Docker Compose pentru a orchestra mai multe containere independente. Fiecare componentă rulează în

propriul container, cu dependențe și configurații izolate.

Figura 8.1 prezintă structura containerizată a aplicației.

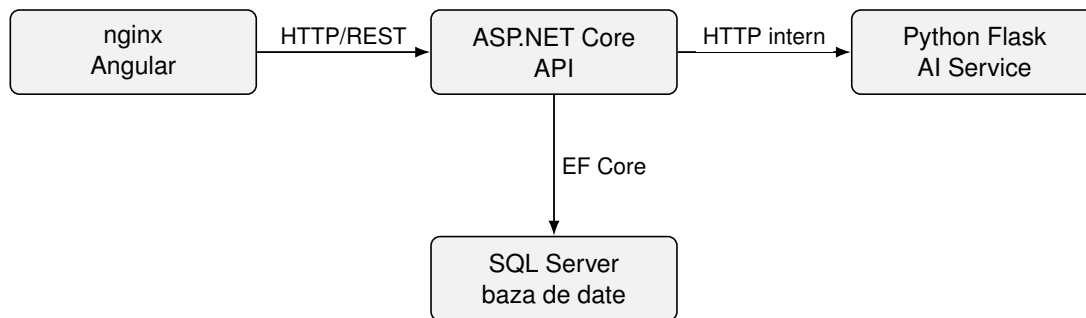


Figura 8.1: Arhitectura containerizată a aplicației WhereWeFishin

Serviciul frontend compilează aplicația Angular în fișiere statice și le servește prin nginx, care acționează și ca reverse proxy pentru cererile API, eliminând problemele CORS în producție. Serviciul api rulează backend-ul .NET și depinde de serviciul de bază de date: Docker Compose garantează că baza de date este pregătită înainte de pornirea API-ului, prin configurarea ordinii de dependențe și a health check-urilor. Serviciul ai-service rulează independent și expune un singur endpoint de inferență pe rețeaua internă. Serviciul database folosește imaginea oficială SQL Server pe Linux.

Modelul antrenat este montat ca volum în containerul Python, fără a fi inclus în imaginea Docker. Această decizie reduce dimensiunea imaginii și permite actualizarea modelului fără a reconstrui containerul.

```
1 services:
2   frontend:
3     build: ./Frontend
4     ports: ["80:80"]
5     depends_on: [api]
6
7   api:
8     build: ./Backend
9     environment:
10      - ASPNETCORE_ENVIRONMENT=Production
11      - ConnectionStrings__Default=${DB_CONNECTION}
12      - Jwt__Key=${JWT_SECRET}
13      - Jwt__Issuer=${JWT_ISSUER}
14     depends_on:
15       database:
16         condition: service_healthy
17
18   ai-service:
19     build: ./PythonService
20     volumes:
21       - ./models/model.pt:/app/model.pt:ro
22
23   database:
24     image: mcr.microsoft.com/mssql/server:2022-latest
25     environment:
26       - SA_PASSWORD=${DB_PASSWORD}
27       - ACCEPT_EULA=Y
28     healthcheck:
29       test: ["CMD", "/opt/mssql-tools/bin/sqlcmd",
```

```

30         "-S", "localhost", "-U", "sa",
31         "-P", "${DB_PASSWORD}", "-Q", "SELECT 1"]
32     interval: 10s
33     retries: 5

```

Listing 8.4: Structura serviciilor în docker-compose.yml

8.7 Gestionarea configurației pe medii de execuție

ASP.NET Core implementează un sistem de configurare ierarhic, în care valorile sunt suprascriere succesive. Fișierul `appsettings.json` conține configurația de bază, comună tuturor mediilor: structura secțiunilor, valorile implicite și referințele la variabilele de mediu. Fișierul `appsettings.Development.json` adaugă configurații specifice mediului local: URL-ul bazei de date locale, chei de test pentru Stripe și o valoare secretă JWT dedicată dezvoltării. Pe mediul de producție, valorile sensibile sunt furnizate exclusiv prin variabile de mediu, cu prioritate maximă față de fișierele de configurare.

Variabila de mediu `ASPNETCORE_ENVIRONMENT` controlează comportamentul aplicației. Pe mediul de dezvoltare, Swagger UI este activ, logarea este detaliată și CORS acceptă orice origine. Pe mediul de producție, Swagger este dezactivat, logarea este restricționată la evenimente importante și CORS permite doar originile cunoscute ale frontendului.

Aceeași logică se aplică și serviciului Python: comportamentul acestuia este diferit în funcție de variabila `FLASK_ENV`. În modul debug, erorile sunt afișate detaliat și reîncărcarea automată a codului este activă. În producție, serverul rulează cu Gunicorn, care gestionează mai eficient conexiunile concurente decât serverul de dezvoltare Flask.

8.8 Sinteza capitolului

Securitatea și deployment-ul nu sunt aspecte secundare ale unui proiect software: ele definesc condițiile în care funcționalitățile implementate pot fi folosite în siguranță și la scară. WhereWeFishin implementează securitate la mai multe niveluri: autentificare JWT cu validare strictă, autorizare pe roluri combinată cu verificări de proprietate a resurselor, rate limiting pe fluxurile sensibile și izolarea componentelor prin containerizare. Gestionarea configurației pe medii diferite asigură că secretele și comportamentele specifice producției nu ajung în mediul de dezvoltare și invers. Această structură oferă o bază solidă pentru operarea platformei în condiții reale.

Aplicația este expusă public la adresa <https://wherewefishin.uk>. Infrastructura de deployment rulează pe un server personal, ceea ce înseamnă că disponibilitatea platformei nu este garantată continuu: serverul poate fi oprit în anumite intervale, iar aplicația să nu fie accesibilă la momentul verificării.

Capitolul 9

Concluzii și perspective de dezvoltare

Lucrarea de față a urmărit proiectarea și dezvoltarea unei aplicații web care să răspundă unor nevoi reale din domeniul pescuitului recreativ. Ideea de bază a fost construirea unei platforme care să nu se limiteze la afișarea locurilor de pescuit sau la păstrarea unui jurnal de capturi, ci să reunească descoperirea locațiilor, rezervarea, administrarea operațională și analiza automată a fișierelor media. Rezultatul este aplicația WhereWeFishin, o soluție multi-rol cu interfață modernă, backend separat și un microserviciu dedicat analizei imaginilor și videoclipurilor.

Pe parcursul lucrării am analizat fundamentele teoretice și tehnologice, am descris cerințele și deciziile de proiectare, am prezentat implementarea componentelor principale și am evaluat comportamentul sistemului prin teste. Lucrarea nu se rezumă la descrierea unei idei, ci prezintă un sistem software concret, funcțional și extensibil.

9.1 Concluzii generale

Se poate aprecia că obiectivul general al lucrării a fost atins. Aplicația oferă un cadru unitar pentru activități care, în practică, sunt adesea gestionate separat sau informal. Utilizatorul poate găsi locuri de pescuit pe hartă, face rezervări, urmărește activitatea în platformă și încarcă fișiere media pentru analiză. Managerul și angajatul au la dispoziție funcții prin care administrează resursele și verifică sesiunile de pescuit.

Un aspect important este că soluția nu a fost gândită pentru un singur tip de utilizator. Platforma deservește simultan clienți, personal operațional și administratori, fiecare cu nevoi și permisiuni distincte. Aceasta a impus o modelare mai atentă a cerințelor, a autorizării și a fluxurilor de lucru, dar a rezultat și într-o aplicație mai apropiată de un scenariu real.

Integrarea componentei de inteligență artificială este un alt punct important. Prin microserviciul separat pentru analiza imaginilor și videoclipurilor, proiectul depășește sfera unei aplicații clasice de rezervări. Detectia peștilor și identificarea speciilor adaugă valoare practică și arată că aplicația poate valorifica conținutul media al utilizatorilor, nu doar datele introduse manual.

Arhitectura separată pe componente s-a dovedit o alegere potrivită. Împărțirea în frontend, backend, bază de date și microserviciu Python a permis dezvoltarea mai clară a modulelor și o integrare mai controlată a funcțiilor costisitoare, cum este procesarea video. Aceasta oferă și o bază bună pentru extinderea ulterioară.

Din perspectivă tehnică, implementarea a demonstrat că o arhitectură cu compo-

nente separate funcționează eficient chiar și la nivel de prototip. Izolarea microserviciului de procesare video a permis utilizarea unor biblioteci specializate, precum PyTorch, OpenCV și FFmpeg, fără a afecta comportamentul și stabilitatea API-ului principal. Aceeași separare va ușura și înlocuirea sau actualizarea modelului de detecție pe măsură ce setul de date crește.

Infrastructura de testare construită în jurul backend-ului reflectă o abordare matură față de calitatea codului. Cele 349 de teste trecute nu sunt un simplu indicator numeric, ci rezultatul unei structuri deliberate: validare la nivelul DTO-urilor, teste de controller pentru contractele API și teste de integrare pentru fluxurile complete. Această structură permite adăugarea de noi funcții cu un risc mai mic de regresii neobservate.

9.2 Contribuțiile principale ale lucrării

Contribuțiile principale pot fi sintetizate în câteva direcții. Prima este construirea unei platforme care combină coerent funcții de explorare, rezervare și administrare. A doua este modelul de roluri care separă clar responsabilitățile utilizatorului obișnuit, ale angajatului, ale managerului și ale administratorului. A treia este componenta geospațială relevantă: harta nu este doar decorativă, ci reprezintă un instrument central pentru descoperirea și gestionarea locurilor de pescuit.

O contribuție importantă este și integrarea serviciului de analiză media bazat pe YOLO și ByteTrack. Această zonă a presupus construirea unui flux complet: de la upload în interfață, prin API, spre serviciul Python și înapoi cu rezultatele stocate și afișate utilizatorului. Nu a fost vorba doar de folosirea unor biblioteci existente, ci de integrarea lor controlată în arhitectura aplicației.

Nu în ultimul rând, infrastructura de testare relevantă pentru backend este un punct forte al proiectului. Rezultatul de 349 de teste trecute în momentul evaluării arată că logica de business a fost tratată și din perspectiva validării regulate, nu doar a implementării vizibile. Structura în mai multe niveluri, anume validare DTO, teste de controller și teste de integrare, oferă o plasă de siguranță reală pentru dezvoltarea ulterioară.

O contribuție mai puțin vizibilă, dar la fel de importantă, este setul de date propriu pentru antrenarea modelului AI. Absența peștilor din seturile de date publice generice a impus colectarea și adnotarea manuală a imaginilor, adaptate la speciile și condițiile fotografice specifice pescuitului recreativ local. Aceasta este o componentă de cercetare aplicată, nu doar de integrare de biblioteci existente.

9.3 Limitele actuale ale soluției

Ca orice proiect de această complexitate, WhereWeFishin are și limite care trebuie recunoscute. Acoperirea testării automate este mult mai bună în backend decât în frontend și în serviciul Python. Interfața și o parte din comportamentul procesării media sunt verificate în principal prin testare manuală și observații directe.

Componenta de analiză video depinde de condițiile în care sunt obținute fișierele media. Calitatea imaginilor, unghiul de filmare, lumina și claritatea apei influențează rezultatul. Performanța modelului nu trebuie interpretată ca absolută, ci în raport cu contextul real de utilizare, limita normală a sistemelor de viziune artificială.

O altă limită este că aplicația, în forma actuală, este optimizată mai ales pentru un scenariu coerent de utilizare și mai puțin pentru un volum mare de utilizatori și operații simultane. Platforma este bine structurată și pregătită pentru extindere, dar unele direcții (cum ar fi procesarea asincronă cu cozi de mesaje sau raportarea statistică avansată) rămân pentru etape ulterioare.

Un aspect operațional care nu a fost abordat este scalabilitatea orizontală: în configurația actuală, aplicația rulează ca o singură instanță per componentă. Extinderea la scenarii cu volum ridicat de cereri simultane ar necesita ajustări la nivelul arhitecturii de deployment: balansare de sarcină, procesare asincronă a analizelor media prin cozi de mesaje și strategii de caching mai avansate pentru răspunsurile frecvent accesate.

9.4 Perspective de dezvoltare

Proiectul poate fi extins în mai multe direcții utile.

O primă direcție este creșterea acoperirii automate a testelor din frontend și a celor pentru serviciul Python. O suită mai echilibrată ar aduce mai multă siguranță la adăugarea de funcții noi și ar reduce riscul de regresii nedetectate timpuriu.

O a doua direcție este îmbunătățirea componentei de analiză media. Pe termen scurt, aceasta presupune extinderea setului de date cu specii suplimentare și imagini în condiții variate de iluminare și unghi. Pe termen mediu, modelul ar putea deveni un instrument util și pentru identificarea speciilor protejate, contribuind la un pescuit recreativ mai responsabil. Rafinarea statisticilor returnate, cum ar fi rapoarte pe specii, comparații între sesiuni sau istorice filtrabile după locație și perioadă, ar crește semnificativ valoarea percepută a funcției de analiză.

O a treia direcție este extinderea zonei de administrare și raportare. Managerii ar putea beneficia de grafice de ocupare a pontoanelor pe intervale de timp, frecvența rezervărilor și distribuția recenziilor. Administratorul platformei ar putea avea instrumente mai bune pentru monitorizarea sistemului și analiza evoluției bazei de utilizatori.

O a patra direcție vizează integrările externe: sincronizarea cu calendare populare, trimiterea de notificări push pentru confirmări și remindere, sau exportul rezervărilor în formate standard. Aceste funcții nu afectează arhitectura de bază, dar adaugă comoditățile pentru utilizatorii frecvenți ai platformei.

Se poate lua în calcul și consolidarea comportamentului de tip PWA: funcționalitate parțial offline, sincronizare în background și confirmări simplificate pentru angajați la verificarea sesiunilor pe teren, în zone cu conectivitate mobilă limitată.

9.5 Încheiere

WhereWeFishin este o aplicație complexă, dar coerentă, care răspunde unei probleme reale printr-o combinație echilibrată de tehnologii web, modelare de domeniu și integrare AI. Pornind de la o nevoie practică bine definită, am construit o soluție modernă utilă atât pescarului obișnuit, cât și administratorului locației.

Realizarea proiectului a presupus navigarea unor compromisuri tehnice care nu apar în aplicații simple: când să tratezi o operație sincron și când asincron, cum să separi responsabilitățile fără cuplare excesivă, cum să echilibrezi securitatea cu ușurința utilizării și când să delegi o funcție unui serviciu extern în loc să o implementezi

intern. Aceste decizii nu au o soluție unică corectă: tocmai de aceea reflectă judecată inginerească, nu doar aplicarea unui tipar cunoscut.

Un aspect care s-a dovedit esențial pe parcurs este rolul testării în menținerea coerenței sistemului. Pe măsură ce platforma a crescut în complexitate, cu mai multe roluri, mai multe fluxuri interdependente și mai multe integrări externe, suita de teste a funcționat ca plasă de siguranță la fiecare modificare. Experiența aceasta confirmă că infrastructura de testare nu este un cost suplimentar față de implementare, ci o condiție pentru controlul calității pe termen lung. Un sistem fără teste nu poate fi extins cu încredere; un sistem cu o suită solidă poate.

Integrarea componentei de inteligență artificială a adus o perspectivă diferită față de celelalte module. Antrenarea modelului, calibrarea pragurilor de inferență și gestionarea ciclului de viață asincron al analizei nu urmează aceleași tipare ca un endpoint REST sau un formular de rezervare. Natura non-deterministă a unui model de detecție, al cărei rezultat depinde de date și nu de logică, a necesitat o arhitectură capabilă să absoarbă această variabilitate fără a compromite predictibilitatea restului aplicației. Separarea explicită a serviciului Python de API-ul principal a fost tocmai mecanismul care a permis acest lucru.

Platforma construită nu este un prototip conceptual, ci un sistem funcțional, cu autentificare și autorizare reale, rezervări corelate cu plată, analiză media prin model antrenat pe date proprii și deployment containerizat expus public. Faptul că sistemul rulează în producție și nu doar pe mașina de dezvoltare reprezintă un criteriu important al maturității tehnice atinse. Această dimensiune practică, de la prima cerere HTTP până la containerul Docker care servește traficul real, este poate cel mai concret indicator al reușitei proiectului.

Dincolo de rezultatul tehnic, lucrarea a reprezentat un exercițiu complet de analiză, proiectare, integrare și validare. Tocmai această combinație dintre fundamentarea teoretică și implementarea practică îi dă consistență. Dacă privim platforma nu ca pe o lucrare finalizată, ci ca pe o primă versiune dintr-un produs viu, atunci pașii următori, anume extinderea modelului AI cu noi specii, scalabilitatea orizontală și suita de teste frontend, nu sunt concluzii ale proiectului, ci puncte de start pentru continuare. Această deschidere este, în sine, un semn al solidității arhitecturii construite.

La început am spus că totul a pornit de la o întrebare simplă: de ce un lucru atât de frumos trebuie să fie atât de complicat de rezervat? Răspunsul la acea întrebare este această platformă. Nu pentru că a rezolvat toate problemele, ci pentru că o problemă reală merită un răspuns pe măsură. Dacă un singur pescar va ajunge mai ușor la baltă datorită ei, fără telefoane inutile și drumuri degeaba, atunci proiectul și-a atins scopul cel mai important. Și poate că, datorită acestei platforme, undeva, cineva se va trezi cu noaptea-n cap plin de același entuziasm cu care mă trezeam și eu.

Bibliografie

- [1] Json web token (jwt). <https://datatracker.ietf.org/doc/html/rfc7519>, 2015. RFC 7519. Accesat la 03 aprilie 2026.
- [2] Owasp top ten. <https://owasp.org/www-project-top-ten/>, 2021. Proiect de referință pentru securitatea aplicațiilor web. Accesat la 09 mai 2026.
- [3] Anglr. <https://anglr.com/>, 2025. Pagină oficială a platformei. Accesat la 19 decembrie 2025.
- [4] Fishangler. <https://www.fishangler.com/>, 2025. Pagină oficială a platformei. Accesat la 17 decembrie 2025.
- [5] Fishbrain. <https://fishbrain.com/>, 2025. Pagină oficială a platformei. Accesat la 14 decembrie 2025.
- [6] Angular. <https://angular.dev/>, 2026. Documentație oficială Angular. Accesat la 08 ianuarie 2026.
- [7] Asp.net core documentation. <https://learn.microsoft.com/en-us/aspnet/core/>, 2026. Documentație oficială Microsoft pentru aplicații și servicii web ASP.NET Core. Accesat la 11 ianuarie 2026.
- [8] Docker documentation. <https://docs.docker.com/>, 2026. Documentație oficială Docker. Accesat la 14 aprilie 2026.
- [9] Entity framework core documentation. <https://learn.microsoft.com/en-us/ef/core/>, 2026. Documentație oficială Microsoft pentru Entity Framework Core. Accesat la 04 martie 2026.
- [10] Ffmpeg documentation. <https://ffmpeg.org/documentation.html>, 2026. Documentație oficială FFmpeg. Accesat la 20 martie 2026.
- [11] Flask documentation. <https://flask.palletsprojects.com/>, 2026. Documentație oficială pentru framework-ul Flask. Accesat la 17 martie 2026.
- [12] Leaflet. <https://leafletjs.com/>, 2026. Bibliotecă open-source pentru hărți interactive. Accesat la 23 ianuarie 2026.
- [13] Opencv documentation. <https://docs.opencv.org/>, 2026. Documentație oficială pentru biblioteca OpenCV. Accesat la 19 martie 2026.

- [14] Pytorch documentation. <https://pytorch.org/docs/stable/>, 2026. Documentație oficială PyTorch. Accesat la 17 aprilie 2026.
- [15] Sql server documentation. <https://learn.microsoft.com/en-us/sql/sql-server/>, 2026. Documentație oficială Microsoft pentru SQL Server. Accesat la 04 martie 2026.
- [16] Stripe documentation. <https://docs.stripe.com/>, 2026. Documentație oficială pentru integrarea plăților online. Accesat la 12 februarie 2026.
- [17] Ultralytics yolo documentation. <https://docs.ultralytics.com/>, 2026. Documentație oficială pentru ecosistemul Ultralytics YOLO. Accesat la 16 februarie 2026.
- [18] xunit.net documentation. <https://xunit.net/>, 2026. Framework open-source pentru teste unitare în .NET. Accesat la 28 aprilie 2026.
- [19] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [20] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [21] Glenford J. Myers, Corey Sandler, and Tom Badgett. *The Art of Software Testing*. Wiley, 3 edition, 2011.
- [22] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, 8 edition, 2014.
- [23] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016.
- [24] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. McGraw-Hill Education, 7 edition, 2019.
- [25] Ian Sommerville. *Software Engineering*. Pearson, 10 edition, 2015.
- [26] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack: Multi-object tracking by associating every detection box. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022.